

In each case, check your answers with your computer system.

- 12. Perform grayscale erosions, dilations, openings, and closings on the cameraman image using the above structuring elements.
- 13.
 - (a) Obtain the grayscale version of the twins image as was used in Chapter 8.
 - (b) Apply salt and pepper noise to the image.
 - (c) Attempt to remove the noise using morphological techniques.
 - (d) Compare the results to median filtering.
- 14. Repeat the above question but use Gaussian noise.
- 15. Use morphological methods to find the edges of the image `stairs.png`. Do this in two ways:
 - (a) By thresholding first and using binary morphology.
 - (b) By using grayscale morphology and thresholding second.

Which seems to give the best results?

- 16. Compare the results of the previous question with standard edge detection techniques.

11 Image Topology

11.1 Introduction

We are often interested in only the very basic aspects of an image: the number of occurrences of a particular object; whether there are holes, and so on. The investigation of these fundamental properties of an image is called *digital topology* or *image topology* and in this chapter we shall investigate some of the more elementary aspects of this subject.

For example, consider an image thresholded and cleaned up with a morphological opening to show a collection of blobs. Such an image can be obtained with MATLAB or Octave by:

```
>> p = imread('pinenuts.png');  
>> p2 = ~im2bw(p,graythresh(p));  
>> p3 = imopen(p2,strel('disk',5));
```

MATLAB/Octave

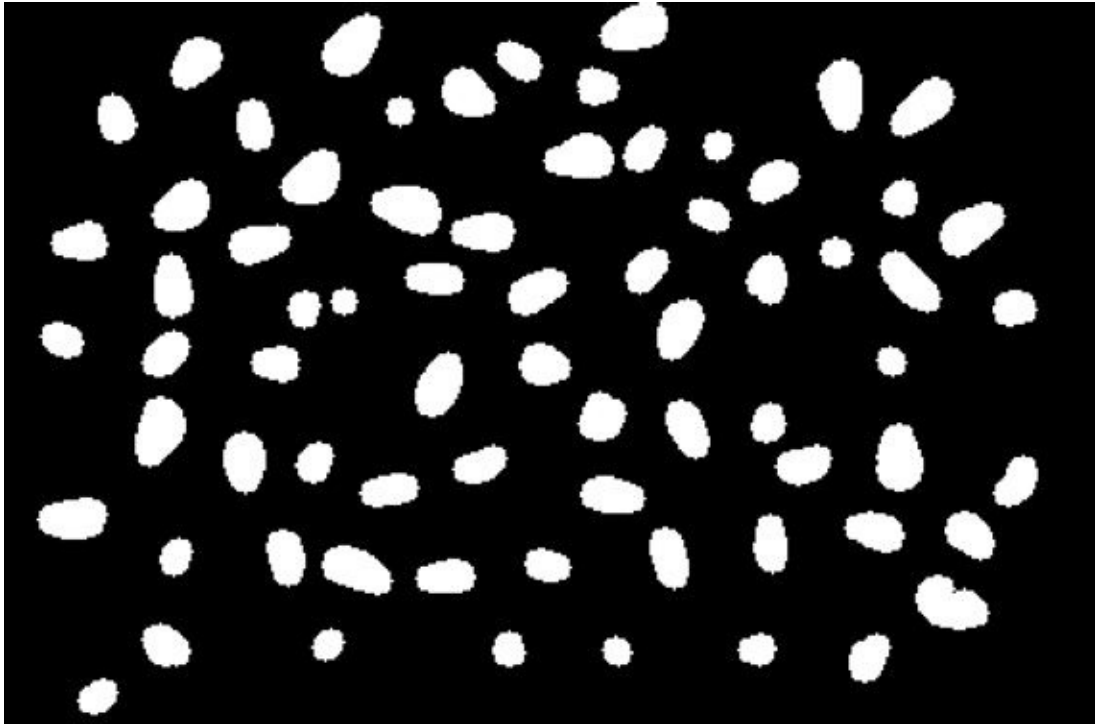
or with Python by:

```
In : import skimage.morphology as mo  
In : p = io.imread('pinenuts.png')  
In : p2 = 1.0-(p>fl.threshold_otsu(p))  
In : p3 = mo.opening(p2,mo.disk(5))
```

Python

The image `n2` is shown in Figure 11.1. The number of blobs can be determined by morphological

Figure 11.1: How many blobs?

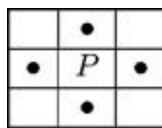


methods. However, the study of image topology provides alternative, and very powerful, methods for such tasks as object counting. Topology provides very rigorous definitions for concepts such as adjacency and distance. As we shall see, skeletonization can be performed very efficiently using topological methods.

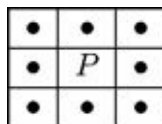
11.2 Neighbors and Adjacency

A first task is to define the concepts of adjacency: under what conditions a pixel may be regarded as being “next to” another pixel. For this chapter, the concern will be with binary images only, and so we will be dealing only with positions of pixels.

A pixel P has four *4-neighbors*:



and eight *8-neighbors*:

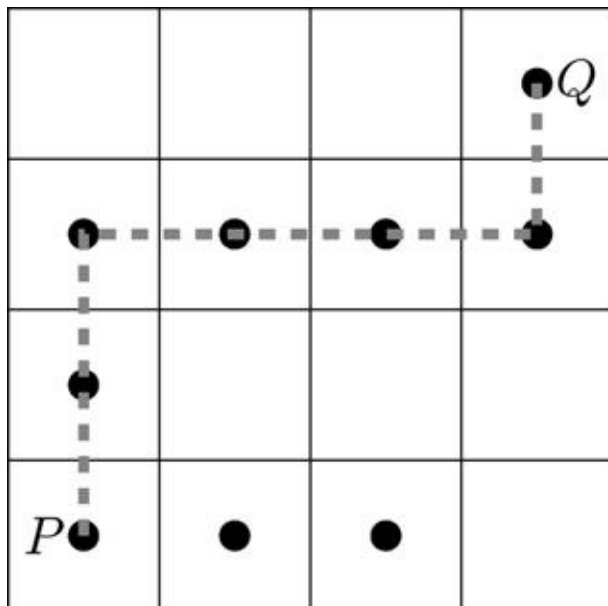


Two pixels P and Q are *4-adjacent* if they are 4-neighbors of one another, and *8-adjacent* if they are 8-neighbors of one another.

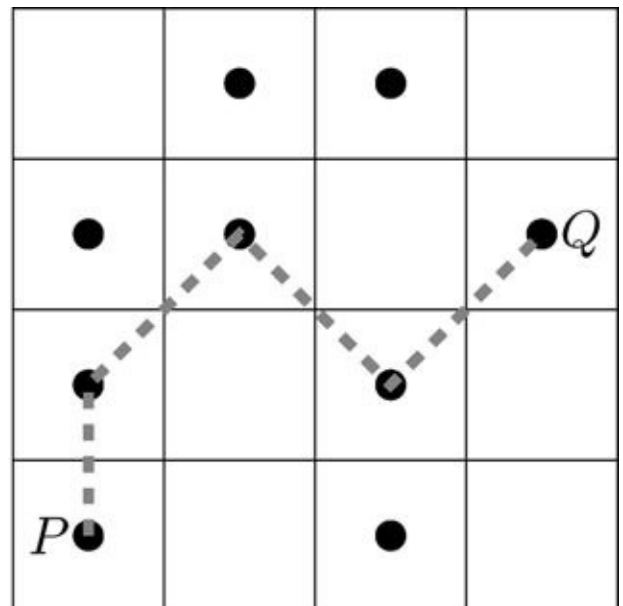
11.3 Paths and Components

Suppose that P and Q are any two (not necessarily adjacent) pixels, and suppose that P and Q can be joined by a sequence of pixels as shown in Figure 11.2.

Figure 11.2: Topological connectedness



(a) 4-connectedness



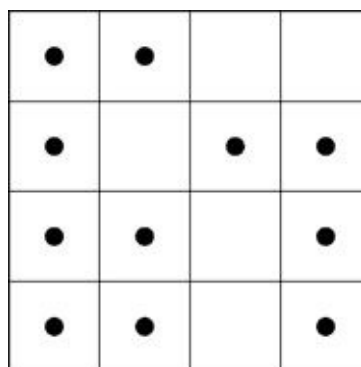
(b) 8-connectedness

If the path contains only 4-adjacent pixels, as the path does in diagram (a), then P and Q are 4-connected. If the path contains 8-adjacent pixels (as shown in (b)), then P and Q are 8-connected.

A set of pixels all of which are 4-connected to each other is called a *4-component*; if all the pixels are 8-connected, the set is an *8-component*.

For example, the following image show in Figure 11.3 has two 4-components (one component containing all the pixels in the left two columns, the other component containing all the pixels in the right two columns), but only one 8-component.

Figure 11.3: Components depend on connectedness



We can define *path* more formally as follows:

A *4-path* from P to Q is a sequence of pixels

$$P = p_0, p_1, p_2, \dots, p_n = Q$$

such that for each $i = 0, 1, \dots, n - 1$, pixel p_i is 4-adjacent to pixel p_{i+1} .

An *8-path* is where the pixels in the sequence connecting P and Q are 8-adjacent.

11.4 Equivalence Relations

A relation $x \sim y$ between two objects x and y is an *equivalence relation* if the relation is

1. *reflexive*; $x \sim x$ for all x
2. *symmetric*; $x \sim y \Leftrightarrow y \sim x$ for all x and y
3. *transitive*; if $x \sim y$ and $y \sim z$ then $x \sim z$ for all x, y and z

Some examples:

1. Numeric equality: $x \sim y$ if x and y are two numbers for which $x = y$.
2. Divisors: $x \sim y$ if x and y are two numbers which have the same remainder when divided by 7.
3. Set cardinality: $S \sim T$ if S and T are two sets with the same number of elements.
4. Connectedness: $P \sim Q$ if P and Q are two connected pixels.

Here are some relations that are not equivalence relations:

1. Personal relations. Define $x \sim y$ if x and y are two people who are related to each other. This is not an equivalence relation. It is reflexive (a person is certainly related to himself or herself); symmetric; but not transitive (can you give an example?).
2. Pixel adjacency. This is not transitive.
3. Subset relation. Define $S \sim T$ if $S \subseteq T$. This is reflexive (a set is a subset of itself); transitive; but not reflexive: if $S \subseteq T$, then it is not necessarily true that $T \subseteq S$.

The importance of the equivalence relation concept is that it allows us a very neat way of dealing with issues of connectedness. We need another definition: an *equivalence class* is the set of all objects equivalent to each other.

We can now define the components of a binary image as being the equivalence classes of the connectedness equivalence relation.

11.5 Component Labeling

In this section we give an algorithm for labeling all the 4-components of a binary image, starting at the top left and working across and down. If p is the current pixel, let u be its upper 4-neighbor, and l its left-hand 4-neighbor:

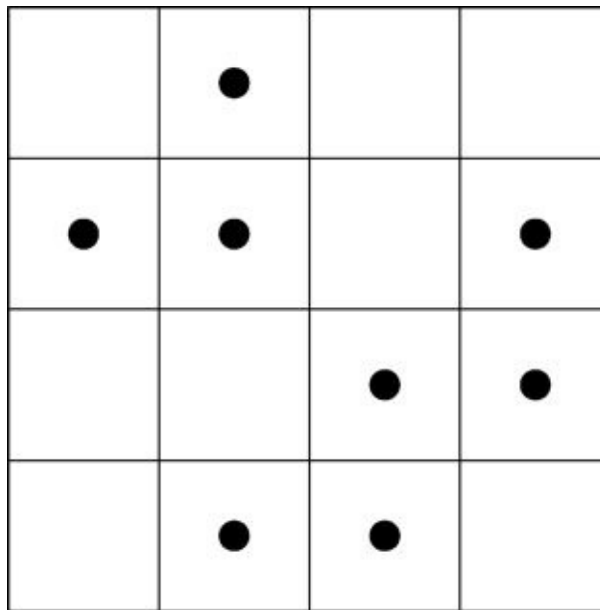
	u	
l	p	

We will “scan” the image row by row moving across from left to right. We will assign labels to pixels in the image; these labels will be the numbers of the components of the image.

For descriptive purposes, a pixel in the image will be called a *foreground pixel*; a pixel not in the image will be called a *background pixel*. And now for the algorithm:

1. Check the state of p . If it is a background pixel, move on to the next scanning position. If it is a foreground pixel, check the state of u and l . If they are both background pixels, assign a new label to p . (This is the case when a new component has been encountered.)
 - If just *one* of u or l is a foreground pixel, then assign its label to p .
 - If both of u and l are foreground pixels and have the same label, assign that label to p .
 - If both of u and l are foreground pixels but have different labels, assign either of those labels to p , and make a note that those two labels are equivalent (since u and l belong to the same 4-component connected through p).
2. At the end of the scan, all foreground pixels have been labeled, but some labels may be equivalent. We now sort the labels into equivalence classes, and assign a different label to each class.
3. Do a second pass through the image, replacing the label on each foreground pixel with the label assigned to its equivalence class in the previous step.

Figure 11.4: An example for the connectedness algorithm

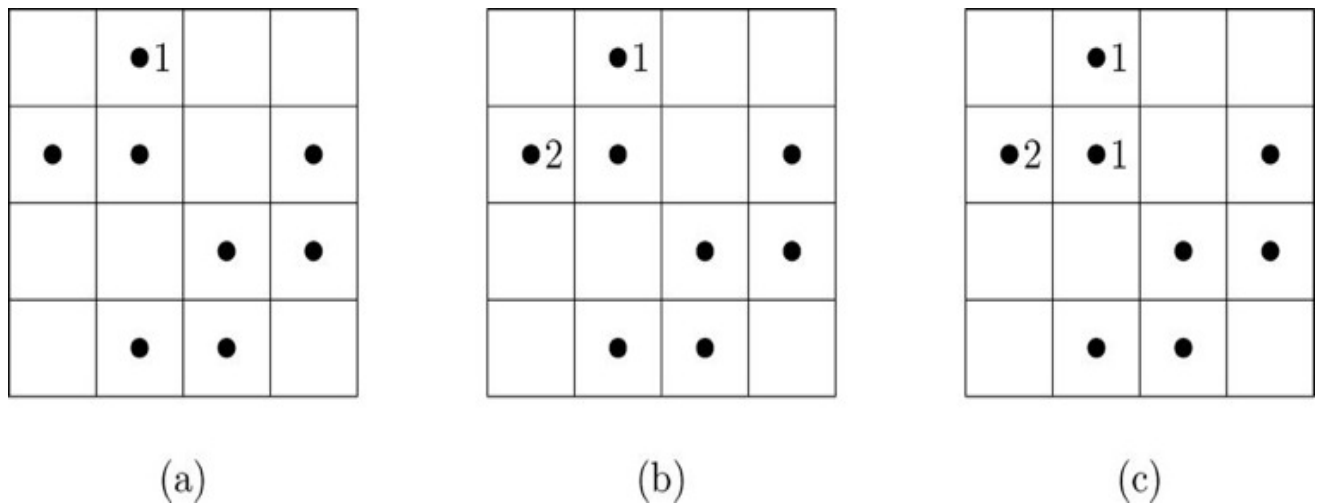


We now give an example of this algorithm in practice, on the binary image shown in Figure 11.4, which has two 4-components: one the three pixels in the top left, the other the five pixels in the bottom right.

Step 1. We start moving along the top row. The first foreground pixel is the bullet in the second place, and since its upper and left neighbors are either background pixels or non-existent, we assign it the label 1, as shown in Figure 11.5(a). In the second row, the first (foreground) pixel again has its upper or left neighbors either background or non-existent, so we assign it a new label—2, as shown in Figure 11.5(b). The second (foreground) pixel in the second row now has both its upper and left neighbors being foreground pixels. However, they have different labels.

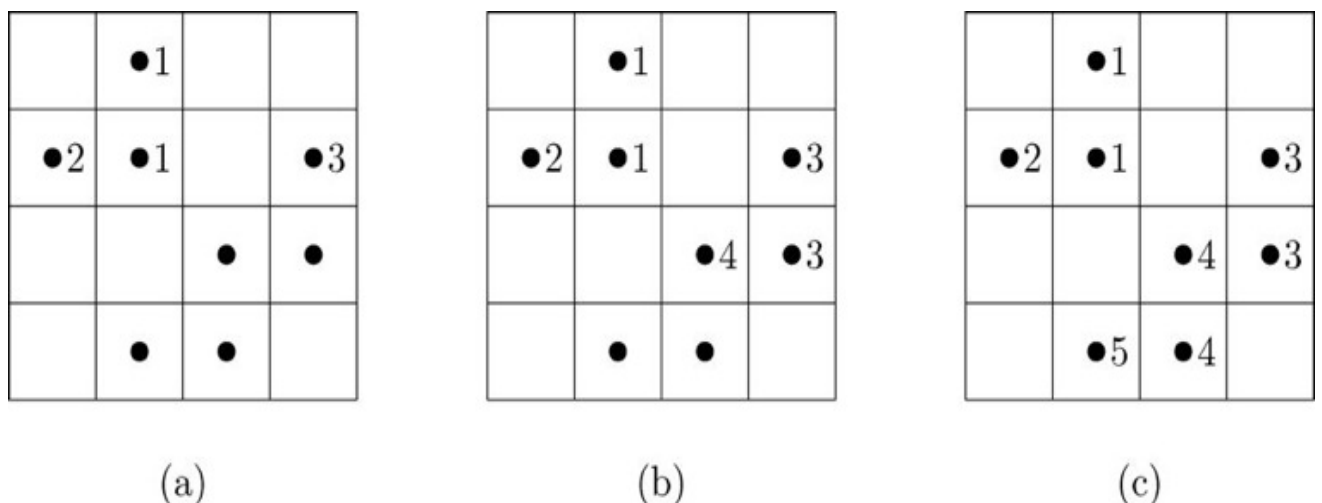
We thus assign either of these labels to this second pixel, say label 1, and make a note that labels 1 and 2 are equivalent. This is shown in Figure 11.5(c).

Figure 11.5: Starting the connectedness algorithm



The third foreground pixel in the second row has both its upper and left neighbors being background pixels, so we assign it a new label—3, as shown in Figure 11.6(a). In the third row, the first foreground pixel has both its upper and left neighbors being background pixels, so we assign it a new label—4. The second (foreground) pixel in the third row now has both its upper and left neighbors being foreground pixels. However, they have different labels. We thus assign either of these labels to this second pixel, say label 3, and make a note that labels 3 and 4 are equivalent. This brings us up to 11.6(b). In the fourth row, the first foreground pixel has both its upper and left neighbors being background pixels, so we assign it a new label—5. The second (foreground) pixel in the fourth row now has both its upper and left neighbors being foreground pixels. However, they have different labels. We thus assign either of these labels to this second pixel, say label 4, and make a note that labels 4 and 5 are equivalent, as shown in Figure 11.6(c). This completes Step 1.

Figure 11.6: Continuing the connectedness algorithm



Step 2. We have the following equivalence classes of labels: $\{1, 2\}$ and $\{3, 4, 5\}$. Assign label 1 to the first class, and label 2 to the second class.

Step 3. Each pixel with labels 1 or 2 from the first step will be assigned label 1, and each pixel with labels 3, 4, or 5 from the first step will be assigned label 2, as shown in Figure 11.7.

Figure 11.7: Finishing the connectedness algorithm

	●1		
●1	●1		●2
		●2	●2
	●2	●2	

This completes the algorithm.

The algorithm can be modified to label 8-components of an image, but in Step 1 we need to consider diagonal elements of p :

d	u	e
l	p	

The algorithm is similar to the previous algorithm; Step 1 is changed as follows:

If p is a background pixel, move on to the next scanning position. If it is a foreground pixel, check d , u , e , and l . If they are all background pixels, assign a new label to p . If just one is a foreground pixel, assign its label to p . If two or more are foreground pixels, assign any of their labels to p , and make a note that all their labels are equivalent.

Steps 2 and 3 are as before.

This algorithm is implemented by the `bwlabel` function, and in Python by the `label` function in the `measure` module of `skimage`. We shall give an example on a small image:

```
>> im = zeros(8,8);
>> im(2:4,3:6) = 1;
>> im(5:7,2) = 1;
>> im(6:7,5:8) = 1;
>> im(8,4:5) = 1
```

MATLAB/Octave

```
In : im = np.zeros((8,8))
In : im[1:4,2:6] = 1
In : im[4:7,1] = 1
In : im[5:7,4:8] = 1
In : im[7,3:5] = 1; print im
```

Python

In MATLAB or Octave the commands

```
>> bwlabel(im,4)
>> bwlabel(im,8)
```

MATLAB/Octave

and in Python the commands

```
In : sk.measure.label(im,4,background=0)+1
In : sk.measure.label(im,8,background=0)+1
```

Python

produce these results:

4-connected components:

```
0 0 0 0 0 0 0 0
0 0 2 2 2 2 0 0
0 0 2 2 2 2 0 0
0 0 2 2 2 2 0 0
0 1 0 0 0 0 0 0
0 1 0 0 3 3 3 3
0 1 0 0 3 3 3 3
0 0 0 3 3 0 0 0
```

8-connected components:

```
0 0 0 0 0 0 0 0
0 0 1 1 1 1 0 0
0 0 1 1 1 1 0 0
0 0 1 1 1 1 0 0
0 1 0 0 0 0 0 0
0 1 0 0 2 2 2 2
0 1 0 0 2 2 2 2
0 0 0 2 2 0 0 0
```

To experiment with a real image, let's try to count the number of pinenuts in the image `pinenuts.png`. The image must first be thresholded to obtain a binary image showing only the bacteria, and then `bwlabel` applied to the result. We can find the number of objects simply by finding the largest label produced.

```
>> n = imread('pinenuts.png');
>> n1 = n<125; n1 = imopen(n1,cr);
>> nl = bwlabel(n1);
>> max(nl(:))
```

MATLAB/Octave

or

```
In : n = io.imread('pinenuts.png')
In : n1 = n<124; n1 = mo.binary_opening(n1,cr)
In : (sk.measure.label(n1,4,background=0)+1).max()
```

Python

which produces the result of 78, being the required number.

11.6 Lookup Tables

Lookup tables provide a very neat and efficient method of binary image processing. Consider the 3×3 neighborhood of a pixel. Since there are 9 pixels in this neighborhood, each with two possible states, the total number of different neighborhoods is $2^9 = 512$. Since the output of any binary operation will be either zero or one, a lookup table is a binary vector of length 512, each element representing the output from the corresponding neighborhoods. Note that this is slightly different from the lookup tables discussed in Section 4.4. In that section, lookup tables were applied to single pixel gray values; here they are applied to neighborhoods.

The trick is then to be able to order all the neighborhoods, so that we have a one-one correspondence between neighborhoods and output values. This is done by giving each pixel in the neighborhood a weighting:

1	8	64
2	16	128
4	32	256

The “value” of the neighborhood is obtained by adding up the weights of one-valued pixels. That value is then the index to the lookup table. For example, the following show neighborhoods and their values:

0	1	0
1	1	0
0	0	1

$$\text{Value} = 2 + 8 + 16 + 256 = 282$$

1	0	0
0	1	1
1	1	1

$$\text{Value} = 1 + 4 + 16 + 32 + 128 + 256 = 437$$

To apply a lookup table, we have to make it first. It would be tedious to create a lookup table element by element, so we use the `makelut` function, which defines a lookup table according to a rule. Its syntax is

`makelut(function, n, P1, P2, ...)`

where `function` is a string defining a MATLAB matrix function, `n` is either 2 or 3, and `P1`, `P2` and so on are optional parameters to be passed to the function. We note that `makelut` allows for lookup tables on 2×2 neighborhoods; in this case the lookup table has only $2^4 = 16$ elements.

Suppose we wish to find the 4-boundary of an image. We define a pixel to be a boundary pixel if it is a foreground pixel which is 4-adjacent to a background pixel. So the function to be used in `makelut` is a function that returns one if and only if the central pixel of the 3×3 neighborhood is a boundary pixel:

```
>> bdy = @(x) (x(5)==1) & (x(2)*x(4)*x(6)*x(8)==0)
```

MATLAB/Octave

Note that for this function we are using the single value matrix indexing scheme, so that `x(5)` is the central pixel of a 3×3 matrix `x`, and `x(2)`, `x(4)`, `x(6)`, `x(8)` are the pixels 4-adjacent to the center.

Now we can make the lookup table:

```
>> lut=makelut(f,3);
```

MATLAB/Octave

and apply it to an image:

```
>> c = imread('circles.png');  
>> cw = applylut(c,lut);  
>> imshow(cw)
```

MATLAB/Octave

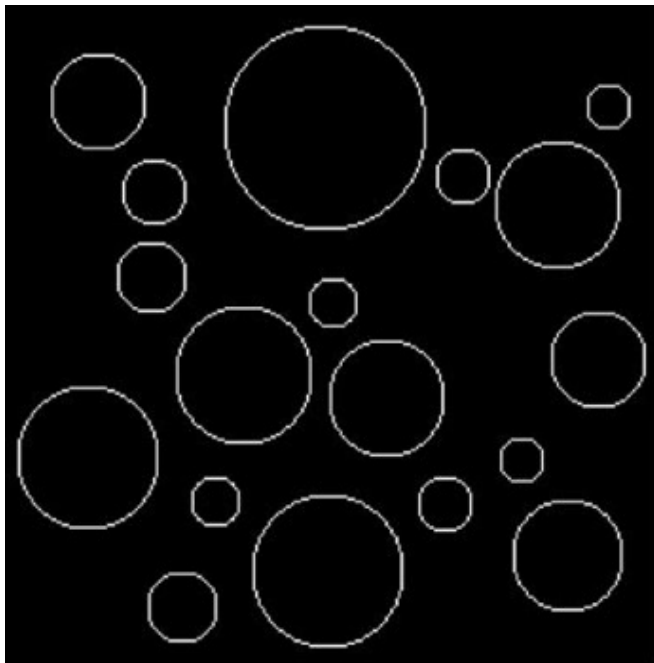
In Python there are no lookup functions as such, but the same affect can be obtained using the `generic_filter` method from the `ndimage` module. Note that the function passed to the filter as its second argument is considered an array with only one row:

```
In : def bdy4(x):  
...:     return ((x[4]==1) and (x[1]*x[3]*x[5]*x[7]==0))*1  
...:  
In : c = io.imread('circles.png').astype('bool')*1  
In : cw = ndi.generic_filter(c,bdy4,size=(3,3))
```

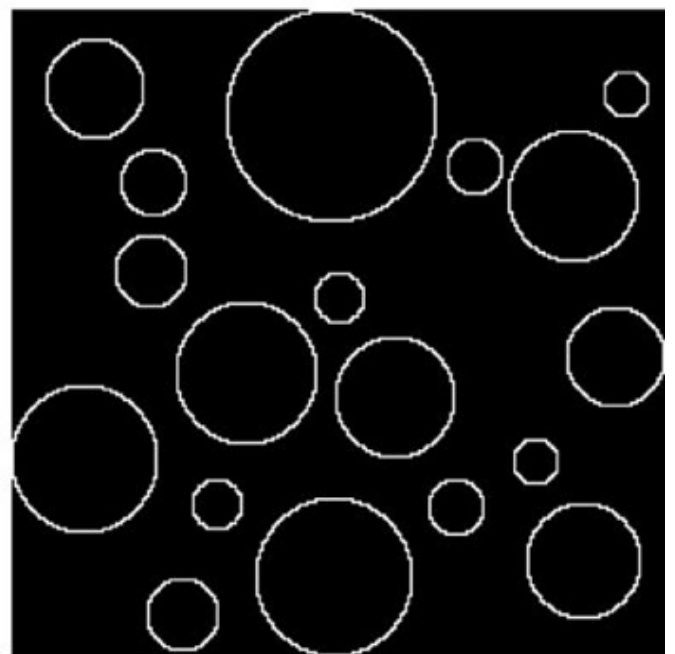
Python

and the result is shown in Figure 11.8(a).

Figure 11.8: Boundaries of circles



(a) 4-boundaries



(b) 8-boundaries

We can easily adjust the function to find the 8-boundary pixels; these being foreground pixels which are 8-adjacent to background pixels:

```
>> bdy8 = @(x) (x(5)==1) & (prod(x(:))==0)
>> lut8 = makelut(bdy8,3);
>> cw8 = applylut(c,lut8);
>> imshow(cw)
```

MATLAB/Octave

or

```
In : def bdy8(x):
...:     return x[4]==1 and x.prod()==0
...:
In : cw8 = ndi.generic_filter(c, bdy8, size=(3,3))
```

Python

and the result is shown in Figure 11.8(b). Note that this is a “stronger” boundary, because more pixels are classified as boundary pixels.

As we shall see in Section 11.8, lookup tables can be used to great effect in performing skeletonization.

11.7 Distances and Metrics

It is necessary to define a function that provides a measure of *distance* between two points x and y on a grid. A distance function $d(x, y)$ is called a *metric* if it satisfies all of:

1. $d(x, y) = d(y, x)$ (symmetry)
2. $d(x, y) \geq 0$ and $d(x, y) = 0$ if and only if $x = y$ (positivity)
3. $d(x, y) + d(y, z) \leq d(x, z)$ (the triangle inequality)

A standard distance metric is provided by *Euclidean distance*, where if $x = (x_1, x_2)$ and $y = (y_1, y_1)$ then

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}.$$

This is just the length of the straight line between the points x and y . It is easy to see that the first two properties are satisfied by this metric: it is always positive, and is zero only when $x_1 = y_1$ and $x_2 = y_2$; that is, when $x = y$. The third property may be proved very easily; it simply says that given three points x , y , and z , then it is shorter to go from x to z directly than via y .

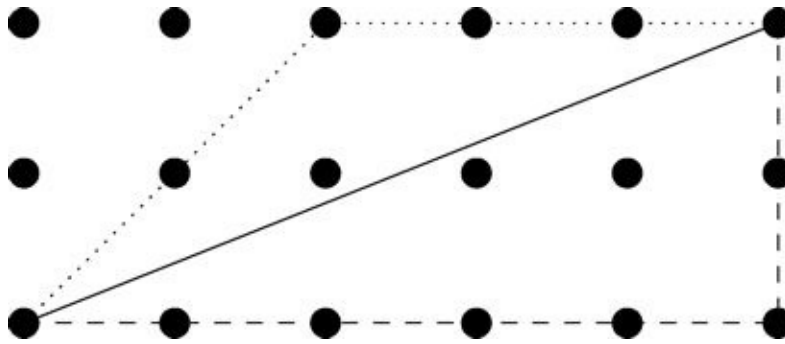
However, if we are constrained to points on a grid, then the Euclidean metric may not be applicable. Figure 11.9 shows the shortest paths for the Euclidean metric, and for 4-connected and 8-connected paths. In this figure, the Euclidean distance is $\sqrt{5^2 + 2^2} \approx 5.39$, and the 4-path (the dashed line) and 8-path (dotted line) have length, measured by the number of line segments in each, of 7 and 5, respectively.

Metrics for measuring distance by 4-paths and 8-paths are given by the following two functions:

$$d_4(x, y) = |x_1 - y_1| + |x_2 - y_2|$$

$$d_8(x, y) = \max\{|x_1 - y_1|, |x_2 - y_2|\}$$

Figure 11.9: Comparison of three metrics



As with the Euclidean distance, the first two properties are immediate; to prove the triangle inequality takes a little effort. The metric d_4 is sometimes known as the *taxicab metric*, as it gives the length of the shortest taxicab route through a rectangular grid of streets.

The Distance Transform

In many applications, it is necessary to find the distance of every pixel from a region R . This can be done using the standard Euclidean distance defined previously. However, this means that to calculate the distance of (x, y) to R , we need to determine all possible distances from (x, y) to pixels in R , and take the smallest value as our distance. This is very computationally inefficient: if R is large, then we have to compute many square roots. A saving can be obtained by noting that since the square root function is increasing, we can write the minimum distance as

$$md(x, y) = \sqrt{\min_{(p, q) \in R} ((x - p)^2 + (y - q)^2)}$$

which involves only one square root. But even this definition is slow to compute. There is a great deal of arithmetic involved, and the finding of a smallest value.

The *distance transform* is a computationally efficient method of finding such distance. We shall describe it in a sequence of steps:

Step 1. Attach to each pixel (x, y) in the image a label $d(x, y)$ giving its distance from R . Start with labeling each pixel in R with 0, and each pixel not in R with ∞ .

Step 2. We now travel through the image pixel by pixel. For each pixel (x, y) replace its label with $\min\{d(x, y), d(x + 1, y) + 1, d(x - 1, y) + 1, d(x, y - 1) + 1, d(x, y + 1) + 1\}$ using $\infty + 1 = \infty$.

Step 3. Repeat Step 2 until all labels have been converted to finite values.

To give some examples of Step 2, suppose we have this neighborhood:

$$\begin{array}{ccc} \infty & \infty & \infty \\ 2 & \infty & \infty \\ \infty & 3 & \infty \end{array}$$

We are only interested in the center pixel (whose label we are about to change), and the four pixels above, below, and to the left and right. To these four pixels we add 1:

$$\begin{array}{ccc} & \infty & \\ 3 & \infty & \infty \\ & 4 & \end{array}$$

The minimum of these five values is 3, and so that is the new label for the center pixel.

Suppose we have this neighborhood:

$$\begin{array}{ccc} 2 & \infty & \infty \\ \infty & \infty & \infty \\ 3 & \infty & 5 \end{array}$$

Again, we add 1 to each of the four pixels above, below, to the left and right, and keep the center value:

$$\begin{array}{ccc} & \infty & \\ \infty & \infty & \infty \\ & \infty & \end{array}$$

The minimum of these values is ∞ , and so at this stage the pixel's label is not changed.

Suppose we take an image whose labels after Step 1 are given below.

$$\begin{array}{cccccc} \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 0 & 0 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & 0 & 0 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty \end{array}$$

Step 1

$$\begin{array}{cccccc} \infty & \infty & 1 & 1 & \infty & \infty \\ \infty & 1 & 0 & 0 & 1 & \infty \\ \infty & \infty & 1 & 0 & 1 & \infty \\ \infty & \infty & 1 & 0 & 1 & \infty \\ \infty & \infty & 1 & 0 & 0 & 1 \\ \infty & \infty & \infty & 1 & 1 & \infty \end{array}$$

Step 2 (first pass)

$$\begin{array}{cccccc} \infty & 2 & 1 & 1 & 2 & \infty \\ 2 & 1 & 0 & 0 & 1 & 2 \\ \infty & 2 & 1 & 0 & 1 & 2 \\ \infty & 2 & 1 & 0 & 1 & 2 \\ \infty & 2 & 1 & 0 & 0 & 1 \\ \infty & \infty & 2 & 1 & 1 & 2 \end{array}$$

Step 2 (second pass)

$$\begin{array}{cccccc} 3 & 2 & 1 & 1 & 2 & 3 \\ 2 & 1 & 0 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 0 & 1 \\ \infty & 3 & 2 & 1 & 1 & 2 \end{array}$$

Step 2 (third pass)

$$\begin{array}{cccccc} 3 & 2 & 1 & 1 & 2 & 3 \\ 2 & 1 & 0 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 1 & 2 \\ 3 & 2 & 1 & 0 & 0 & 1 \\ 4 & 3 & 2 & 1 & 1 & 2 \end{array}$$

Step 2 (final pass)

At this stage we stop, as all label values are finite.

An immediate observation is that the distance values given are not in fact a very good approximation to “real” distances. To provide better accuracy, we need to generalize the above transform. One way to do this is to use the concept of a “mask”; the mask used above was

$$\begin{array}{ccc} & 1 & \\ 1 & 0 & 1 \\ & 1 & \end{array}$$

Step two in the transform then consists of adding the corresponding mask elements to labels of the neighboring pixels, and taking the minimum. To obtain good accuracy with simple arithmetic, the mask will generally consist of integer values, but the final result may require scaling.

Suppose we apply the mask

4	3	4
3	0	3
4	3	4

to the above image. Step 1 is as above; Step 2 is:

∞	4	3	3	4	∞	7	4	3	3	4	7	7	4	3	3	4	7
∞	3	0	0	3	∞	6	3	0	0	3	6	6	3	0	0	3	6
∞	4	3	0	3	∞	7	4	3	0	3	6	7	4	3	0	3	6
∞	∞	3	0	3	∞	8	6	3	0	3	6	8	6	3	0	3	6
∞	∞	3	0	0	3	∞	6	3	0	0	3	9	6	3	0	0	3
∞	∞	4	3	3	4	∞	7	4	3	3	4	10	7	4	3	3	4
Step 2 (first pass)						Step 2 (second)						Step 2 (third)					

at which point we stop, and divide all values by three:

2.3	1.3	1	1	1.3	2.3
2	1	0	0	1	2
2.3	1.3	1	0	1	2
2.7	2	1	0	1	2
3	2	1	0	0	1
3.3	2.3	1.3	1	1	1.3

These values are much closer to the Euclidean distances than those provided by the first transform.

Even better accuracy can be obtained by using the mask

	11		11	
11	7	5	7	11
	5	0	5	
11	7	5	7	11
	11		11	

and dividing the final result by 5:

11	7	5	5	7	11	2.2	1.4	1	1	1.4	2.2
10	5	0	0	5	10	2	1	0	0	1	2
11	7	5	0	5	10	2.2	1.4	1	0	1	2
14	10	5	0	5	7	2.8	2	1	0	1	1.4
16	10	5	0	0	5	3.2	2	1	0	0	1
16	11	7	5	5	7	3.2	2.2	1.4	1	1	1.4
Result of transform						After division by 5					

The method we have described can in fact be very slow; for a large image we may require many passes before all distance labels become finite. A quicker method requires *two* passes only: the first pass starts at the top left of the image, and moves left to right, and top to bottom. The second pass starts at the bottom right of the image, and moves right to left, and from bottom to top. For this method we break the mask up into two halves; one half inspects only values to the left and above (this is used for the first pass), and the other half inspects only values to the right and below (this is used for the second pass).

Such pairs of masks are shown in Figures 11.10 to 11.12, and the solid lines show how the original mask is broken up into its two halves.

We apply these masks as follows: first surround the image with zeros (such as for spatial filtering), so that at the edges of the image the masks have values to work with. For the forward pass, for each pixel at position (i, j) , add the values given in the forward mask to its neighbors and take the minimum, and replace the current label with this minimum.

Figure 11.10: Simple pair of masks for the two-pass distance transform

	1	
1	0	

Forward mask

	0	1
	1	

Backward mask

Figure 11.11: More accurate pair of masks for the two-pass distance transform

4	3	4
3	0	

Forward mask

	0	3
4	3	4

Backward mask

Figure 11.12: Masks that are more accurate still

	11		11	
11	7	5	7	11
	5	0		

Forward mask

		0	5	
11	7	5	7	11
	11		11	

Backward mask

This is similar to Step 2 in the original algorithm, except that we are using less pixels. We do the same for the backward pass, except that we use the backward mask, and we start at the bottom right.

If we apply forward masks 1 and 2 to our image, the results of the forward passes are:

∞	∞	∞	∞	∞	∞
∞	∞	0	0	1	2
∞	∞	1	0	1	2
∞	∞	2	0	1	2
∞	∞	3	0	0	1
∞	∞	4	1	1	2

Use of mask 1

∞	∞	∞	∞	∞	∞
∞	∞	0	0	3	6
∞	4	3	0	3	6
8	7	5	0	3	6
11	8	4	0	0	3
12	8	4	3	3	4

Use of mask 2

After applying the backward masks, we will obtain the distance transforms as above.

Implementing the Distance Transform

We can easily write a function to perform the distance transform using the second method as above. Our function will implement the transform as follows:

1. Using the size of the mask, pad the image with an appropriate number of zeros.
2. Change each zero to infinity, and each one to zero.
3. Create forward and backward masks.
4. Perform a forward pass: replace each label with the minimum of its neighborhood plus the forward mask.
5. Perform a backward pass: replace each label with the minimum of its neighborhood plus the backward mask.

Rather than provide a general function, here is how to apply the masks from Figure 11.11 to a binary image.

Suppose first that the `circles.png` image has been read in to a variable `c`. In MATLAB or Octave, the first three items above can be obtained with:

```
>> c2 = padarray(c,[1,1]);  
>> c2(find(c2==0))=Inf; c2(find(c2==1))=0;  
>> fmask = [4 3 4;3 0 0;0 0 0]; bmask = [0 0 0;0 0 3;4 3 4]
```

MATLAB/Octave

and in Python by

```
In : c2 = np.pad(c,[1,1],mode='constant').astype('float64')  
In : c2[np.where(c2==0)]=float('inf'); c2[np.where(c2==1)]=0  
In : fmask = np.array([[4,3,3],[3,0,0],[0,0,0]])  
In : bmask = np.array([[0,0,0],[0,0,3],[4,3,3]])
```

Python

The forward and backward passes can be performed with nested loops:

```
>> for i = 1:256,  
>   for j = 1:256,  
>     c2(i+1,j+1) = min(min(c2(i:i+2,j:j+2)+fmask));  
>   end  
> end  
  
>> for i = 256:-1:1,  
>   for j = 256:-1:1,  
>     c2(i+1,j+1) = min(min(c2(i:i+2,j:j+2)+bmask));  
>   end  
> end
```

MATLAB/Octave

or in Python with

```
In : for i in range(256):  
...:     for j in range(256):  
...:         c2[i+1,j+1] = (c2[i:i+3,j:j+3]+fmask).min()  
...:  
  
In : for i in range(256)[::-1]:  
...:     for j in range(256)[::-1]:  
...:         c2[i+1,j+1] = (c2[i:i+3,j:j+3]+bmask).min()  
...:
```

Python

The result can be shown as

```
>> imshow(mat2gray(2(2:257,2:257)))
```

MATLAB/Octave

or

```
In : io.imshow(c2[1:257,1:257])
```

Python

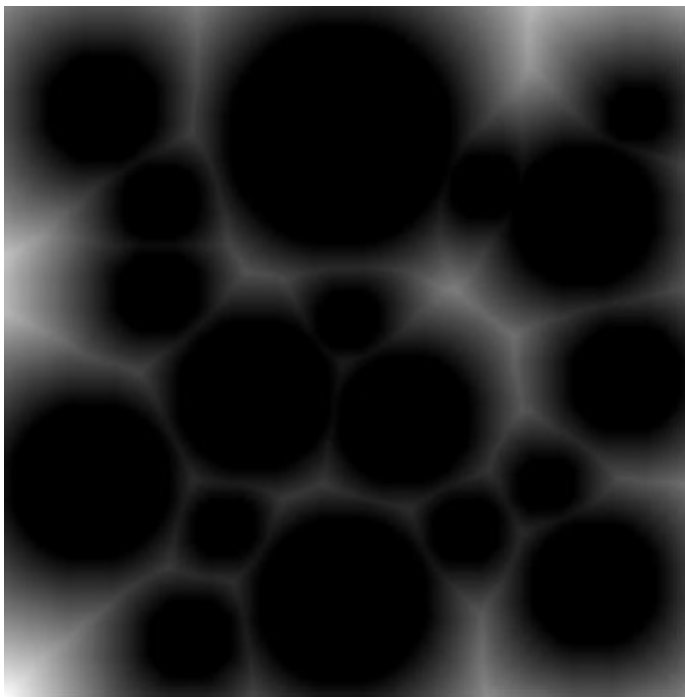
and is shown in Figure 11.13(a). Sometimes better information can be obtained by inverting the image and finding the distance inside the objects, as shown in Figure 11.13(b)

11.8 Skeletonization

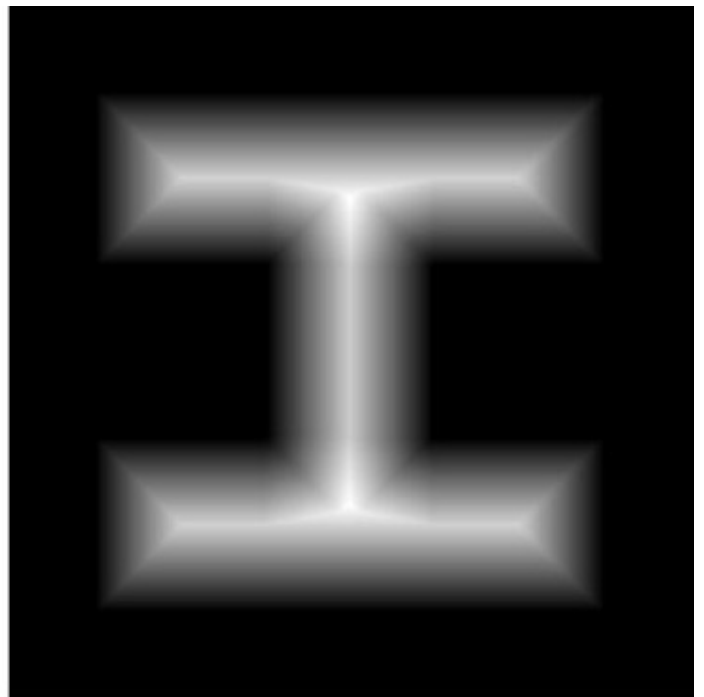
The *skeleton* of a binary object is a collection of lines and curves that encapsulate the size and shape of the object. There are in fact many different methods of defining a skeleton, and for a given object, many different possible skeletons. But to give a general idea, we show a few examples in Figure 11.14.

A problem with skeletonization is that very small changes to an object can result in large changes to the skeleton. Figure 11.15 shows what happens if we take away or add to the central image of Figure 11.14.

Figure 11.13: Examples of a distance transform



(a) “External” distances



(b) “Internal” distances

Figure 11.14: Examples of skeletonization

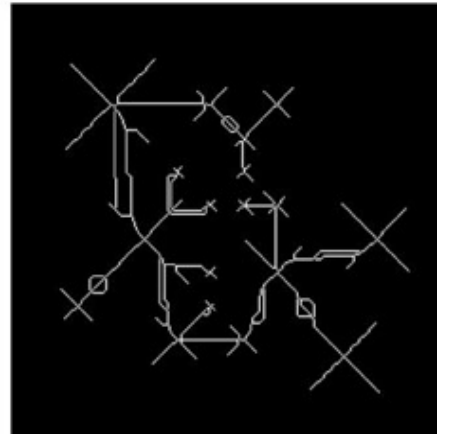
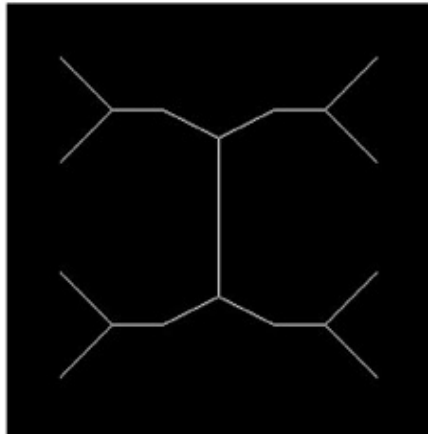
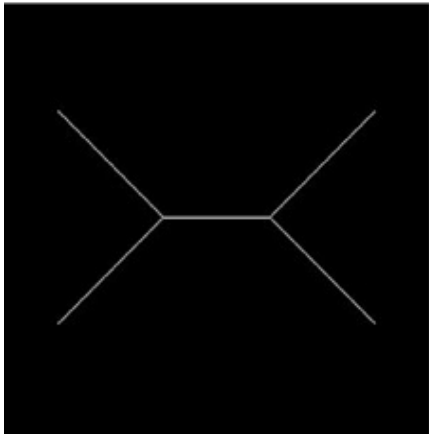
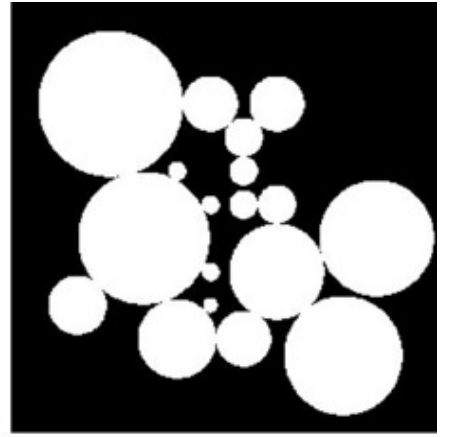
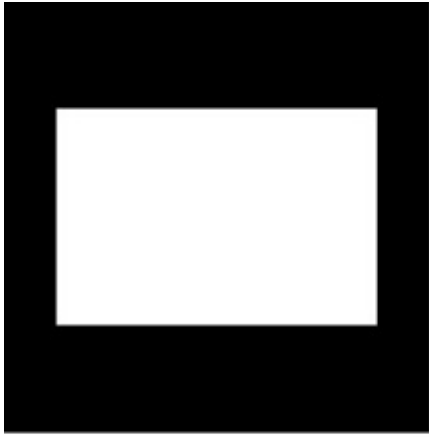
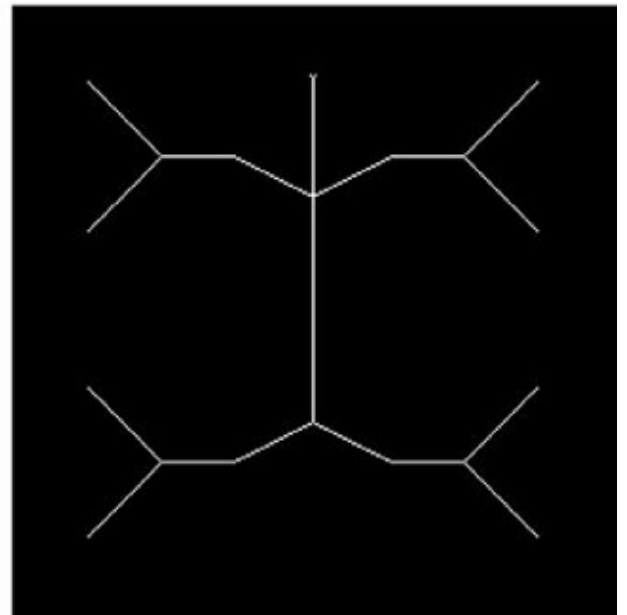
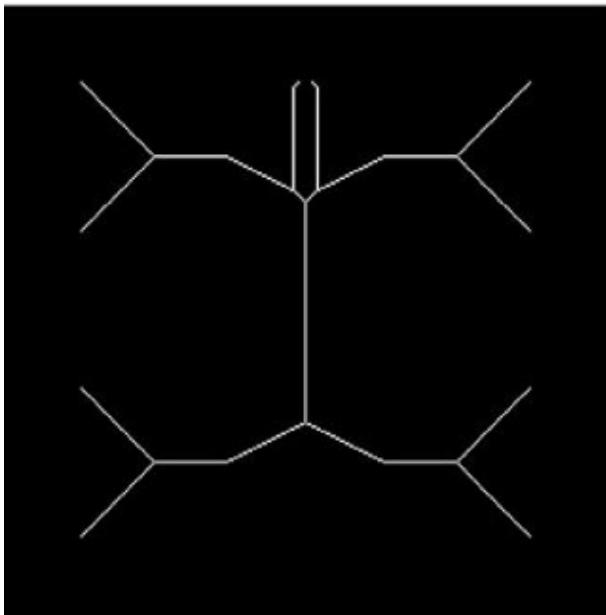
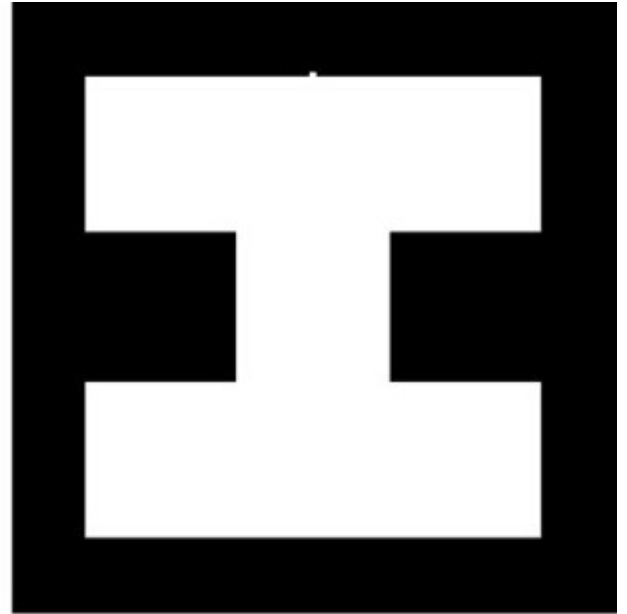


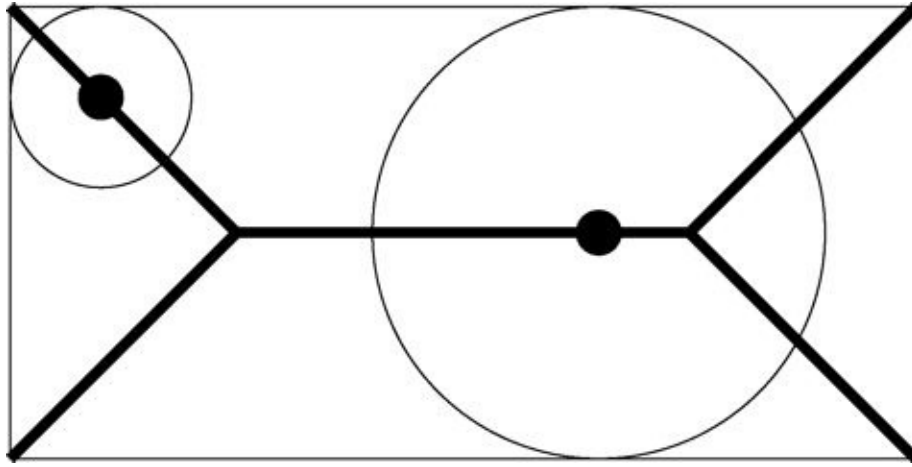
Figure 11.15: Skeletons after small changes to an object



One very popular way of defining a skeleton is by the *medial axis* of an object: a pixel is on the medial axis if it is equidistant from at least two pixels on the boundary of the object. To implement this definition directly requires a quick and efficient method of obtaining approximate distances; one way is to use the distance transform. Other ways of approaching the medial axis are:

- Imagine the object to be burning up by a fire that advances at a constant rate from the boundary. The places where two “lines” of fire meet form the medial axis.
- Consider the set of all circles lying within the object, which touch at least two points on the boundary. The centers of all such circles form the medial axis. Figure 11.16 shows this in action.

Figure 11.16: The medial axis of an object



A very good account of the medial axis (and of skeletonization in general) is given by Parker [32].

Topological methods provide some powerful methods of skeletonization in that we can directly define those pixels which are to be deleted to obtain the final skeleton. In general, we want to delete pixels that can be deleted without changing the *connectivity* of an object, which do not change the number of components, change the number of holes, or the relationship of objects and holes. For example, Figure 11.17 shows a non-deletable pixel; deleting the center (boxed) pixel introduces a hole into the object. In Figure 11.18, there is another example of a non-deletable pixel; in this case, deletion *removes* a hole, in that the hole and the exterior become joined. In Figure 11.19, there is a further example of a non-deletable pixel; in this case, deletion breaks the object into two separate components. Sometimes we need to consider whether the object is 4-connected or 8-connected. In the previous examples this was not a problem. However, look at the examples in Figure 11.20. In Figure 11.20(a), the central point can not be deleted without changing both the 4-connectivity and the 8-connectivity. In Figure 11.20(b) deleting the central pixel will change the 4-connectivity, but not the 8-connectivity. A pixel that can be deleted without changing the 4-connectivity of the object is called *4-simple*; similarly, a pixel that can be deleted without changing the 8-connectivity of the object is called *8-simple*. Thus, the central pixel in Figure 11.20(a) is neither 4-simple nor 8-simple, but the central pixel in Figure 11.20(b) is 8-simple but not 4-simple.

Figure 11.17: A non-deletable pixel: creates a hole

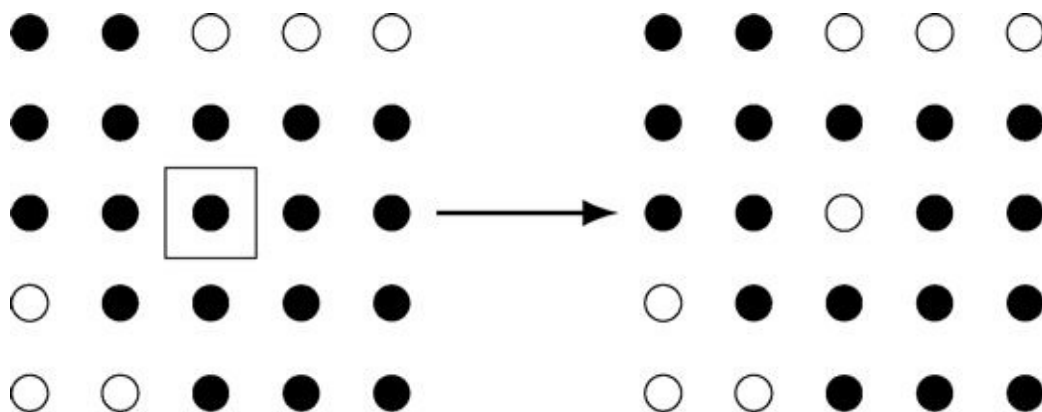


Figure 11.18: A non-deletable pixel: removes a hole

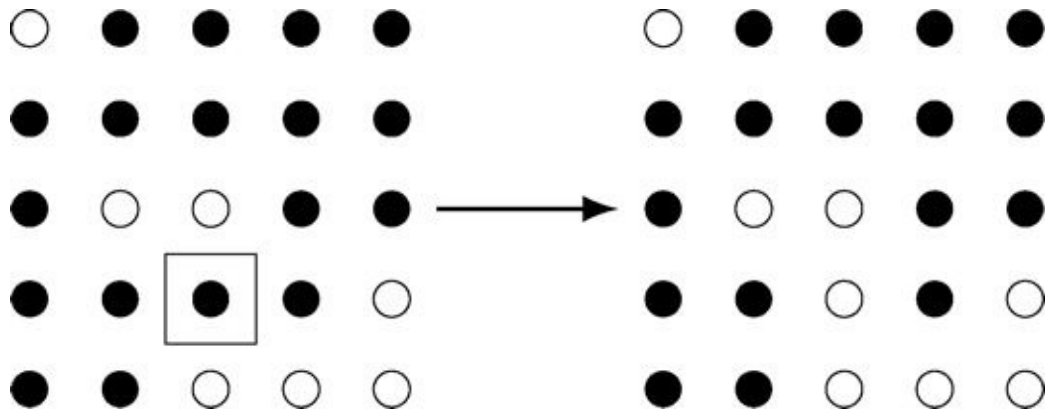
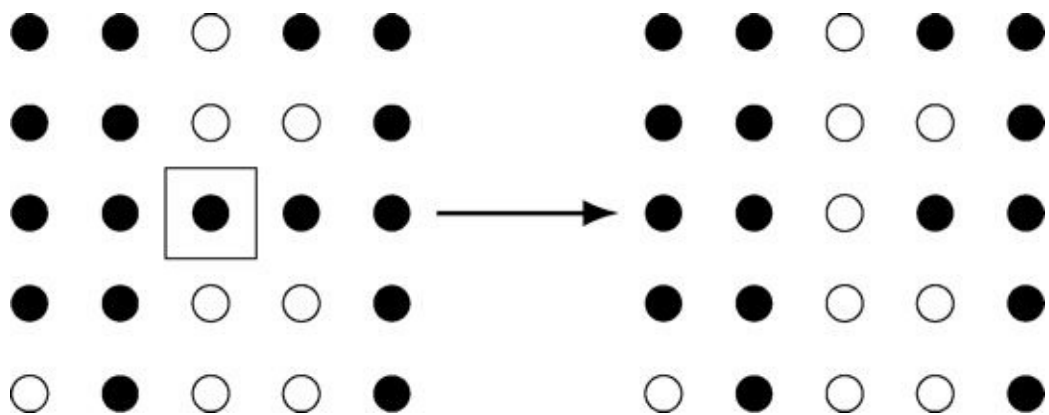
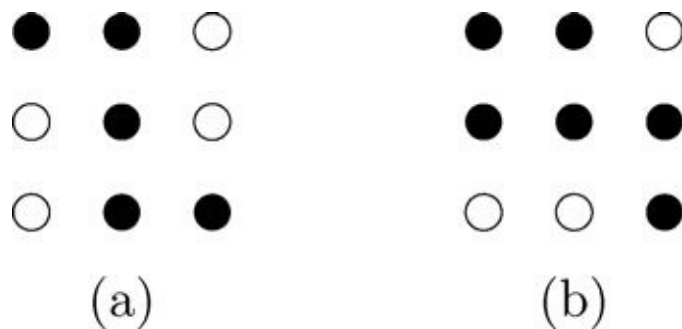


Figure 11.19: A non-deletable pixel: disconnects an object



A pixel can be tested for deletability by checking its 3×3 neighborhood. Look again at Figure 11.20(a). Suppose the central pixel is deleted. There are two options:

Figure 11.20: Are the center points deletable?



1. The top two pixels and the bottom two pixels become separated; in effect breaking up the object.
2. The top two pixels and the bottom two pixels are joined by a chain of pixels outside the shown neighborhood. In this case, all pixels will encircle a hole, and removing the central pixel will remove the hole, as in Figure 11.18.

To check whether a pixel is 4-simple or 8-simple, we introduce some numbers associated with the neighborhood of a foreground pixel p . First define N_p to be the 3×3 neighborhood of p , and N_p^* to be the 3×3 neighborhood *excluding* p . Then:

$A(p)$ = the number of 4-components in N_p^*
 $C(p)$ = the number of 8-components in N_p^*
 $B(p)$ = the number of foreground pixels in N_p^*

For example, in Figure 11.20(a), we have

$$\begin{aligned}
 A(p) &= 2 \\
 C(p) &= 2 \\
 B(p) &= 4
 \end{aligned}$$

and in Figure 11.20(b) we have

$$\begin{aligned}
 A(p) &= 2 \\
 C(p) &= 1 \\
 B(p) &= 5
 \end{aligned}$$

We can see the last example by deleting the central pixel and enumerating the components of the remaining foreground pixels. This is shown in Figure 11.21. Simple points can now be characterized entirely in terms of the values of $A(p)$ and $C(p)$:

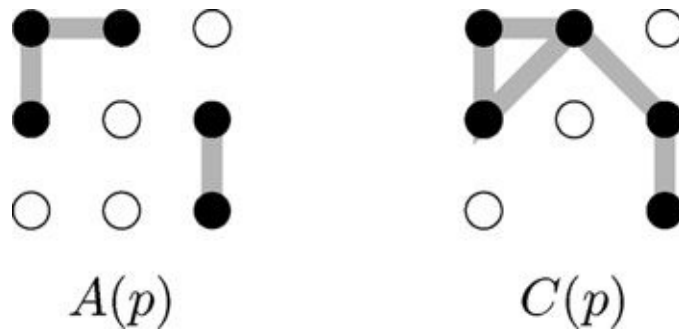


Figure 11.21: Components of N_p^*

be characterized entirely in terms of the values of $A(p)$ and $C(p)$:

A foreground pixel p is 4-simple if and only if $A(p) = 1$, and is 8-simple if and only if $C(p) = 1$.

Returning again to Figure 11.20(b), since $C(p) = 1$ the central pixel p is 8-simple and so can be deleted without affecting the 8-connectivity of the object. But since $A(p) \neq 1$, the central pixel p is *not* 4-simple and so cannot be deleted without affecting the 4-connectivity of the object. This is exemplified in Figure 11.3.

Note that $A(p)$ and $B(p)$ can be computed simply by using the various labeling functions discussed earlier. For example, consider the neighborhood shown in Figure 11.20(b):

```
>> n = [1 1 0;1 1 1;0 0 1]
>> n(5) = 0;
>> max(max(bwlabel(n,4)))
ans =
    2

>> max(max(bwlabel(n,8)))
ans =
    1
```

MATLAB/Octave

Similarly:

```
In : from skimage.measure import label
In : n = np.array([[1,1,0],[1,1,1],[0,0,1]])
In : n[1,1] = 0
In : label(n,4,background=0).max()+1
Out: 2
In : label(n,8,background=0).max()+1
Out: 1
```

Python

How Not To Do Skeletonization

So now we know how to check if a pixel can be deleted without effecting the connectivity of the object. In general, a skeletonization algorithm works by an iteration process: at each step identifying deletable pixels and deleting them. The algorithm will continue until no further deletions are possible.

One way to remove pixels is as follows:

At each step, find all foreground pixels that are 4-simple and delete them all.

Sounds good? Let's try it on a small rectangle of size 2×4 :

```
0  0  0  0  0  0
0  1  1  1  1  0
0  1  1  1  1  0
0  0  0  0  0  0
```

If we check the pixels in this object carefully, we will find that they are *all* 4-simple. Deleting them all will thus remove the object completely: a very undesirable result. Clearly we need to be a bit more careful about which pixels we can delete and when. We need to add an extra test for deletability, so that we do not delete too many pixels.

We have two options here:

1. We can provide a step-wise algorithm, and change the test for deletability at each step.
2. We can apply a different test for deletability according to where the pixel lies on the image grid.

Algorithms that work according to the first option are called *subiteration algorithms*; algorithms that work according to the second option are called *subfield algorithms*.

The Zhang-Suen Skeletonization Algorithm

This algorithm has attained the status of a modern classic; it has some faults (as we shall see), but it works fairly fast, and most of the time produces acceptable results. It is an example of a subiteration algorithm, in that we apply a slightly different deletability test for different steps of the algorithm. In each step, the neighboring pixels are indexed as

$$\begin{array}{ccc} p_1 & p_4 & p_7 \\ p_2 & p_5 & p_8 \\ p_3 & p_6 & p_9 \end{array}$$

where $p_5 = p$ is the pixel being considered for deletion.

Step N

Flag a foreground pixel $p = 1$ to be deletable if

1. $2 \leq B(p) \leq 6$,
2. $X(p) = 1$,
3. If N is odd, then

$$\begin{aligned} p_4 \cdot p_8 \cdot p_6 &= 0 \\ p_8 \cdot p_6 \cdot p_2 &= 0. \end{aligned}$$

If N is even, then

$$\begin{aligned} p_4 \cdot p_8 \cdot p_2 &= 0 \\ p_4 \cdot p_6 \cdot p_2 &= 0. \end{aligned}$$

Delete all flagged pixels.

Continue until there are no more deletable pixels in two successive iterations.

Item 3 can be written alternatively as:

3. If N is odd, then

$$p_6 = 0, \text{ or } p_8 = 0, \text{ or } p_2 = p_4 = 0$$

If N is even, then

$$p_2 = 0, \text{ or } p_4 = 0, \text{ or } p_6 = p_8 = 0.$$

If we check the diagram for the neighbors of a foreground pixel p , we see that we can rephrase this item as:

For odd iterations, delete only pixels that are on the right hand side, or bottom of an object, or on a northwest corner.

For even iterations, delete only pixels that are on the left hand side, or top of an object, or on a southeast corner.

Figure 11.22: Deletion in the Zhang-Suen algorithm

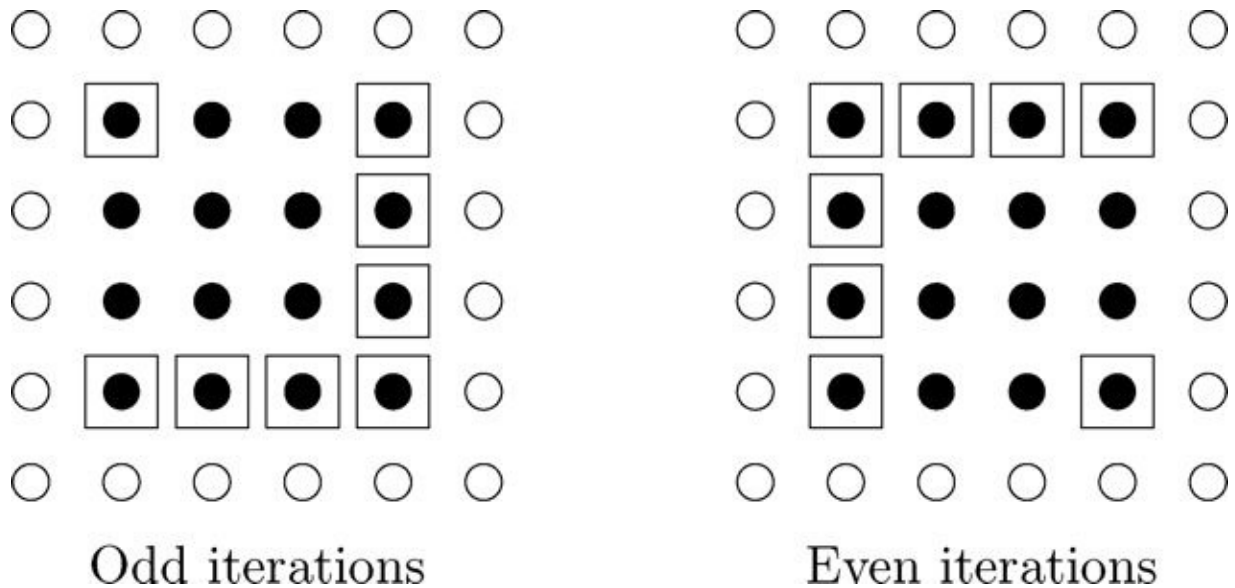


Figure 11.22 shows pixels that may be considered for deletion at different iterations. Item 1 in the algorithm ensures that we do not delete pixels that have only one neighbor, or have seven or more. If a pixel has only one neighbor, it would be at the end of a skeletal line, and we would not want it to be deleted. If a pixel has seven neighbors, then deleting it would start an unacceptable erosion into the object's shape. This item thus ensures that the basic shape of the object is kept by the skeleton. Item 2 is our standard connectivity condition.

A major fault with this algorithm is that there are objects that will be deleted completely. Consider a 2×2 square:

$$\begin{array}{cccc} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{array}$$

We can check carefully that every item is satisfied by every pixel, and hence every pixel will be deleted.

An example. Consider the L shape shown in Figure 11.23, where for ease of viewing we have replaced all the zeros (background pixels) with dots. The boxed pixels show those that will be deleted by Steps 1 and 2 of the algorithm. Figure 11.24 shows Steps 3 and 4 of the skeletonization. After Step 4 no more deletions are possible, and so the skeleton consists of the unboxed foreground pixels in the right-hand diagram of Figure 11.24. Note that the skeleton does not include the corners of the original object.

Implementation in MATLAB and Octave. We can implement this algorithm easily in MATLAB or Octave; we use lookup tables: one for the odd iterations, and one for the even. We then apply these lookup tables alternately until there is no change in the image for two successive iterations. We manage this by keeping three images at any given time: the *current* image, the *previous* image, and the *last* (that is, before the previous) image. If the current and last images are equal, we stop. Otherwise, “push” the images back: the previous image becomes last, and the current image becomes the previous image. We then apply whichever lookup table is appropriate to the current image to create the new current image. That is:

```

    last ← previous
previous ← current
current ← applylut(current, lut)

```

Figure 11.23: Steps 1 and 2 of a Zhang-Suen skeletonization

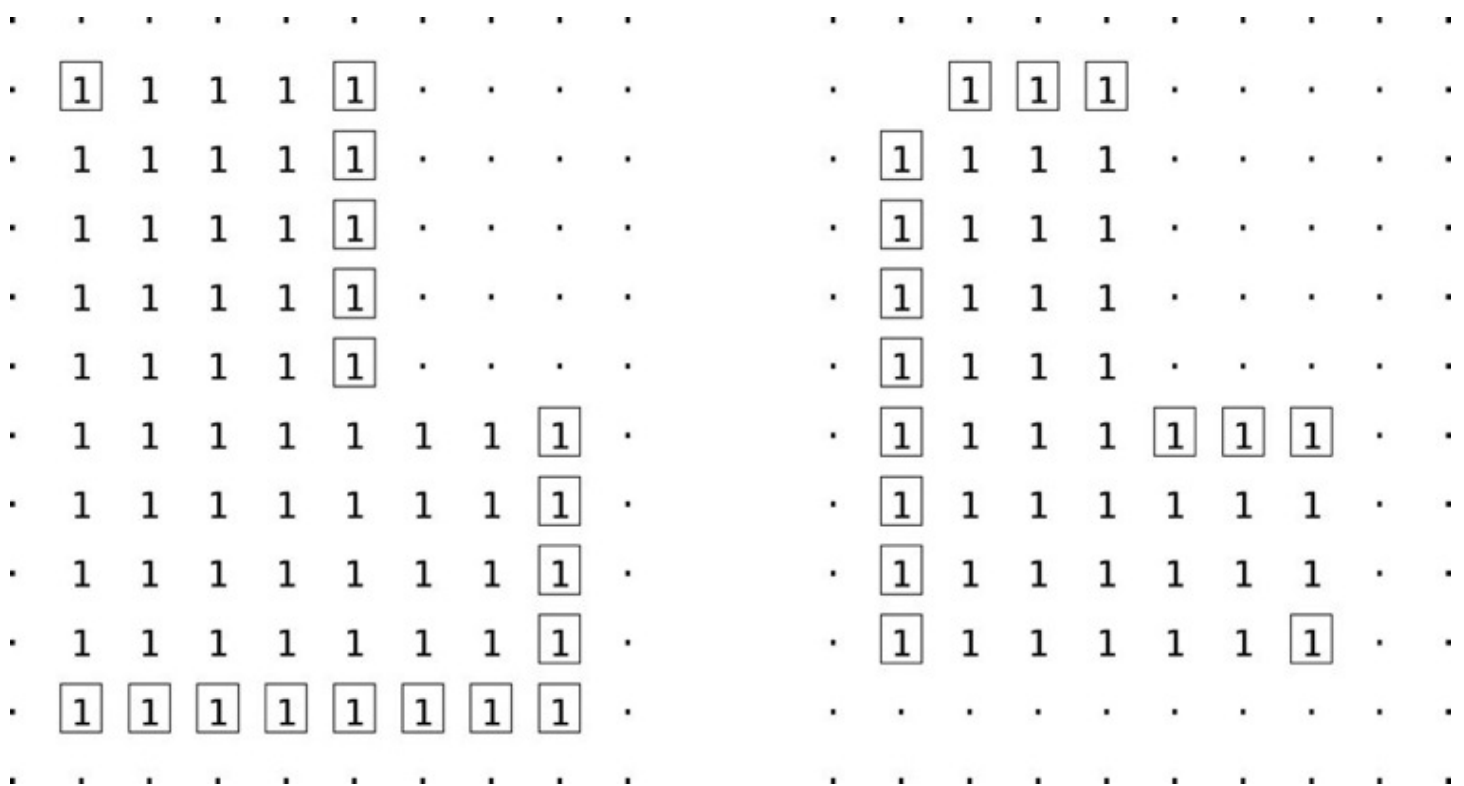
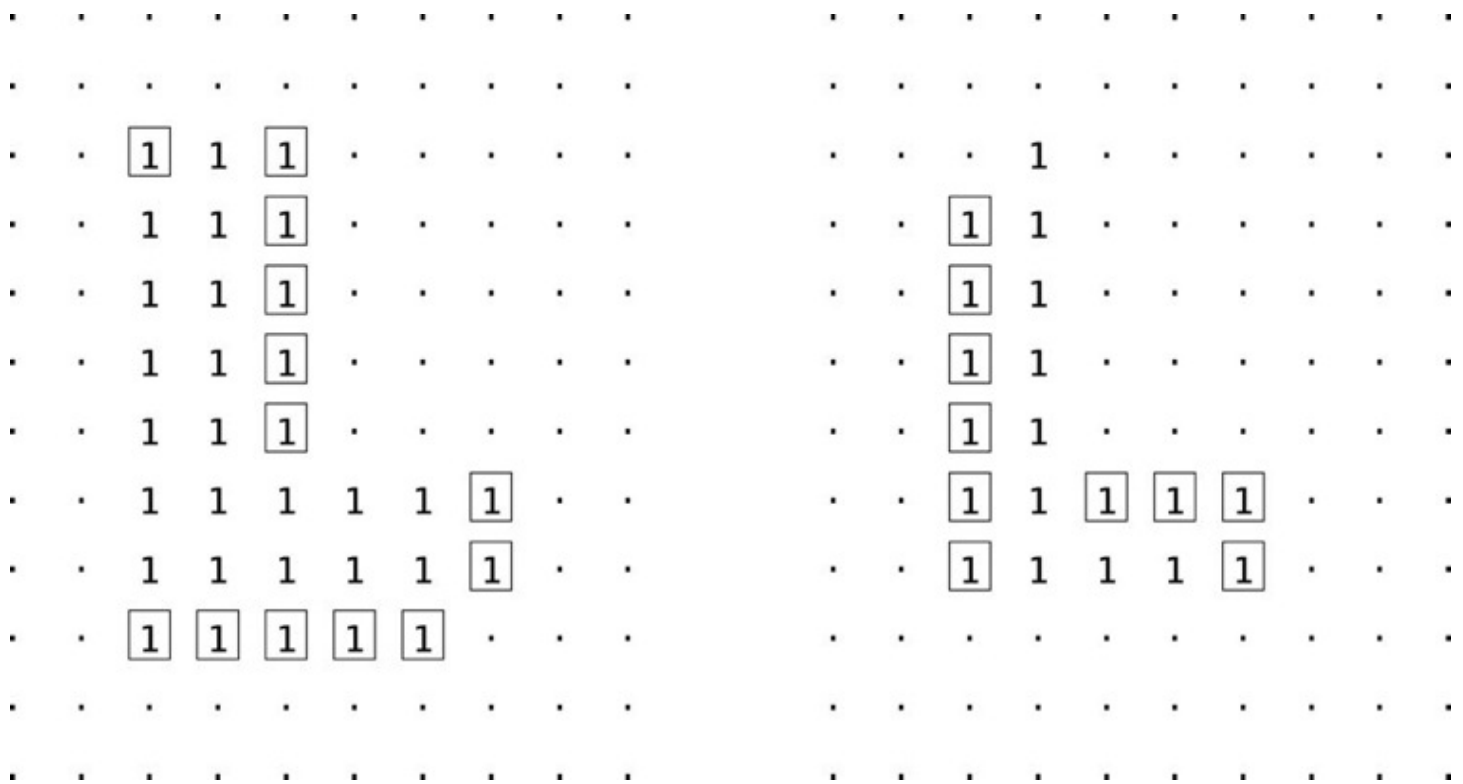


Figure 11.24: Steps 3 and 4 of a Zhang-Suen skeletonization



The lookup table for the odd iterations can be created as:

```
>> function out = zs_odd(x)
>   t = x; t(5)=0; B = sum(t(:)); X=max(max(bwlabel(t,4)));
>   out = (B>=2) & (B<=6) & (X==1);
>   out = out&(t(6)==0 | t(8)==0 | (t(2)==0 & t(4)==0));
>   end
>> lut_odd = makelut(@zs_odd,3);
```

MATLAB/Octave

and for the even iterations as:

```
>> function out = zs_even(x)
>   t = x; t(5)=0; B = sum(t(:)); X=max(max(bwlabel(t,4)));
>   out = (B>=2) & (B<=6) & (X==1);
>   out = out&(t(2)==0 | t(4)==0 | (t(6)==0 & t(8)==0));
>   end
>> lut_even = makelut(@zs_even,3);
```

MATLAB/Octave

Applying these lookup tables to the L shape from the previous figures can be easily done, but to simplify the work we shall perform two iterations at a time:

```

>> L = zeros(12,10); L(2:11,2:6)=1; L(7:11,7:9)=1;
>> previous = L;
>> current = L & ~applylut(L,lut_odd);
>> while true,
>     current = current & ~applylut(current,lut_even);
>     current = current & ~applylut(current,lut_odd);
>     if isequal(previous,current), break;end
>     previous = current;
> end

```

MATLAB/Octave

Note that the results of applying the lookup tables is an array A of zeros and ones, where the ones give the positions of the deletable foreground pixels. To delete these pixels from the current array X we need to perform a set difference $X \setminus A$, which can be implemented with $X \& \sim A$.

At this stage, the value of `current` will be the skeleton shown in Figure 11.24.

Implementation in Python. In fact, the Zhang-Suen algorithm is implemented as the `skeletonize` method in the `skimage.morphology` module. But as an example, we will show how we can do it ourselves. Start with writing a function for both the odd and even iterations, using a separate argument (here called `parity`) to distinguish them:

```

In : def zs_fun(x,parity):
...:     n = np.reshape(x,[3,3])
...:     n[1,1] = 0
...:     B = n.sum()
...:     X = sk.measure.label(n,background=0).max()+1
...:     out = (B>=2) and (B<=6) and (X==1)
...:     if parity==0:
...:         out = out & (x[1]==0 or x[3]==0 or (x[5]==0 and x[7]==0))
...:     elif parity==1:
...:         out = out & (x[5]==0 or x[7]==0 or (x[1]==0 and x[3]==0))
...:     return out

```

Python

The functions will be applied using the `ndimage.generic_filter` method:

```

In : from scipy.ndimage import generic_filter as gfilt
In : L = zeros((12,10)); L[1:11,1:6]=1; L[6:11,6:9]=1;
In : prev = np.copy(L)
In : curr = prev*(1-gfilt(prev,zs_fun,size=(3,3),extra_arguments=(1,)))

```

Python

Then as with MATLAB or Octave we apply the even and odd iterations until there is no more change:

```

In : while True:
...:     curr = curr*(1-gfilt(curr,zs_fun,size=(3,3),extra_arguments=(0,))
...:     )
...:     curr = curr*(1-gfilt(curr,zs_fun,size=(3,3),extra_arguments=(1,))
...:     )
...:     if np.array_equal(prev,curr):
...:         break
...:     else:
...:         prev = np.copy(curr)
...:

```

Python

Two more examples; the circles image and the “nice work” image. Both the images and their skeletonizations are shown in Figure 11.25.

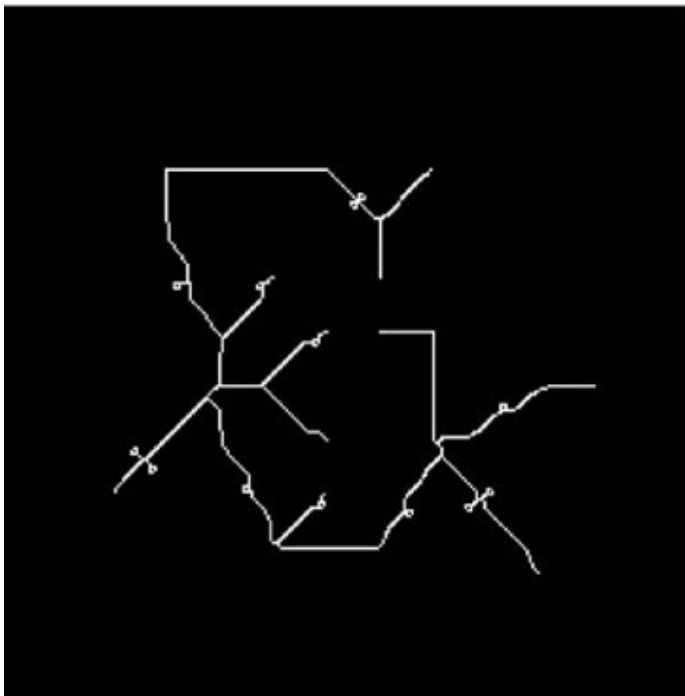
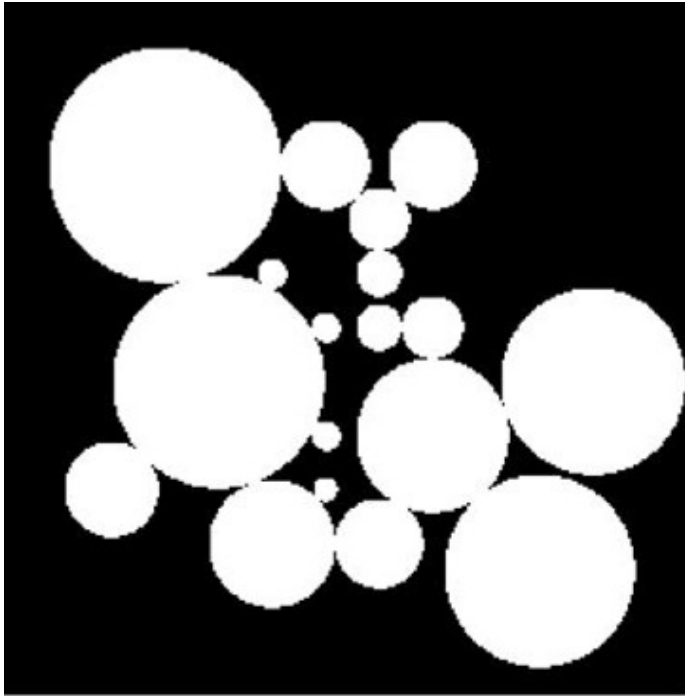
The Guo-Hall Skeletonization Algorithm

There are in fact a number of Guo-Hall algorithms; we will investigate one that has the advantages of being simple to describe, easy to implement, fast to run, and gives good results. What could be better?

The Guo-Hall algorithm is an example of a subfield algorithm. We imagine the image grid being labeled with 1's and 2's in a chessboard configuration:

1	2	1	2	...
2	1	2	1	...
1	2	1	2	...
2	1	2	1	...
⋮	⋮	⋮	⋮	⋱

Figure 11.25: Examples of the Zhang-Suen skeletonization



At Step 1, we only consider foreground pixels whose labels are 1. At Step 2, we only consider foreground pixels whose labels are 2. We continue alternating between pixels labeled 1 and pixels labeled 2 from step to step until no more deletions are possible. Here is the algorithm:

Flag a foreground pixel p as deletable if all of these conditions are met:

1. $C(p) = 1$,
2. $B(p) > 1$,
3. p is 4-adjacent to a background pixel

then delete all flagged pixels. This is done, in parallel, at alternate iterations for each of the two subfields.

Continue until no deletions are possible in two successive iterations.

Consider our “L” example from above. We first superimpose a chessboard of 1's and 2's. In Step 1 we just consider 1's only. Step 1 shown in Figure 11.26 illustrates the first step: we delete only those 1's satisfying the Guo-Hall deletability conditions. These pixels are shown in squares. Having deleted them, we now consider 2's only; the deletable 2's are shown in Step 2.

Figure 11.26: Steps 1 and 2 of a Guo-Hall skeletonization

Flag a foreground pixel p as deletable if all of these conditions are met:

1. $C(p) = 1$,
2. $B(p) > 1$,
3. p is 4-adjacent to a background pixel

then delete all flagged pixels. This is done, in parallel, at alternate iterations for each of the two subfields.

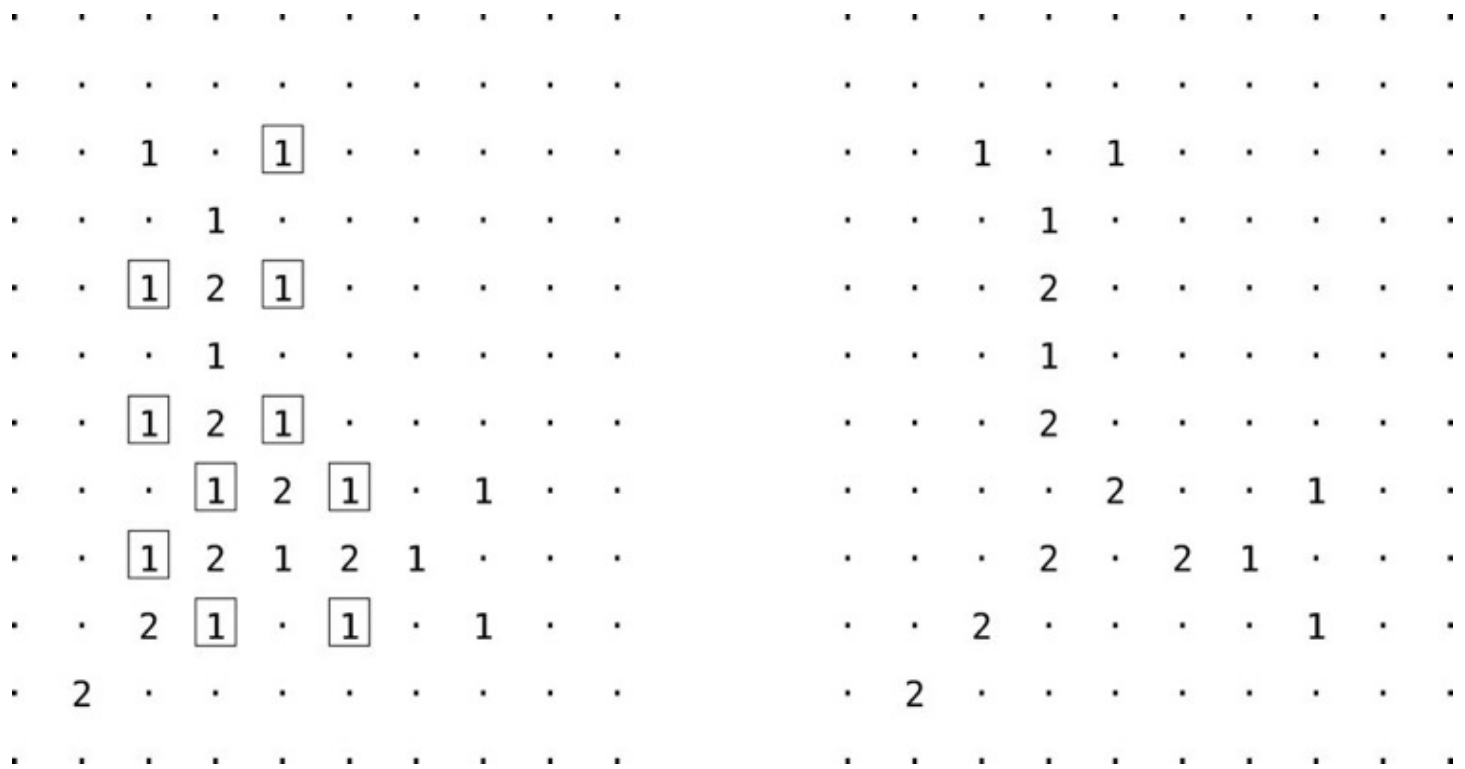
Continue until no deletions are possible in two successive iterations.

Steps 3 and 4 as shown in Figure 11.27 continue the work; by Step 4 there are no more deletions to be done, and we stop. We notice two aspects of the Guo-Hall algorithm as compared with Zhang-Suen:

1. More pixels may be deleted at each step, so we would expect the algorithm to work faster.
2. The final result includes more corner information than the Zhang-Suen algorithm.

We can implement this in using very similar means to our implementation of Zhang-Suen.

Figure 11.27: Steps 3 and 4 of a Guo-Hall skeletonization



Implementation in Python. Start by creating a version of the image tiled with ones and twos. Using the “nice work” image stored as binary array `n`:

```
In : r,c = n.shape
In : n2 = np.tile(np.array([[1,2],[2,1]]),[r/2,c/2])*n
```

Python

As with the Zhang-Suen algorithm, create a function to flag deletable pixels:

```
In : def gh_fun(x,parity):
...:     n = (np.reshape(x,[3,3])>0)*1
...:     n[1,1] = 0
...:     B = n.sum()
...:     X = sk.measure.label(n,8,background=0).max()+1
...:     return ((B>=2) and (X==1) and (x[1]*x[3]*x[5]*x[7]==0) and (x
[4]==parity))
```

Python

Then we can create the current and previous images:

```
In : prev = np.copy(n2)
In : curr = prev*(1-gfilt(prev,gh_fun,size=(3,3),extra_arguments=(1,)))
```

Python

Now apply the algorithm until there are no further changes:

```
In : while True:
...:     curr = curr*(1-gfilt(curr,gh_fun,size=(3,3),extra_arguments=(2,))
...:     )
...:     curr = curr*(1-gfilt(curr,gh_fun,size=(3,3),extra_arguments=(1,))
...:     )
...:     if np.array_equal(prev,curr):
...:         break
...:     else:
...:         prev = np.copy(curr)
...:
```

Python

Implementation in MATLAB and Octave. Start as before by defining the function to apply to the subfields:

```
>> function out = gh(x)
>   n = x; n(5) = 0;
>   B = sum(n(:)); X = max(max(bwlabel(n,8)));
>   S = n(2)*n(4)*n(6)*n(8);
>   out = (B>=2 & X==1 & S==0);
> end
>> gh_lut = makelut(@gh,3);
```

MATLAB/Octave

Next, create a chessboard version of the image:

```
>> [r,c] = size(n);
>> n2 = repmat([1 2;2 1],r/2,c/2).*n;
```

MATLAB/Octave

Now apply the lookup table to the ones and twos in turn, until there is no change:

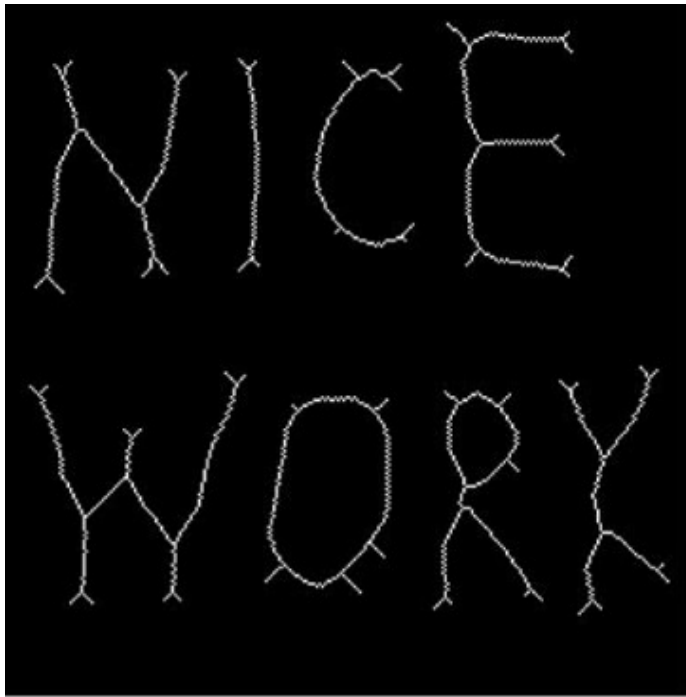
```
>> prev = n2;
>> g = apply(n2>0, gh_lut);
>> curr = prev .* ~((prev==1) & (g==1));
>> while true,
>   g = apply(curr>0, gh_lut);
>   curr = curr .* ~((curr==2) & (g==1));
>   g = applylut(curr>0,gh_lut);
>   curr = curr .* ~((curr==1) & (g==1));
>   if isequal(prev,curr), break;end
>   prev = curr;
> end
>> imshow(curr>0)
```

MATLAB/Octave

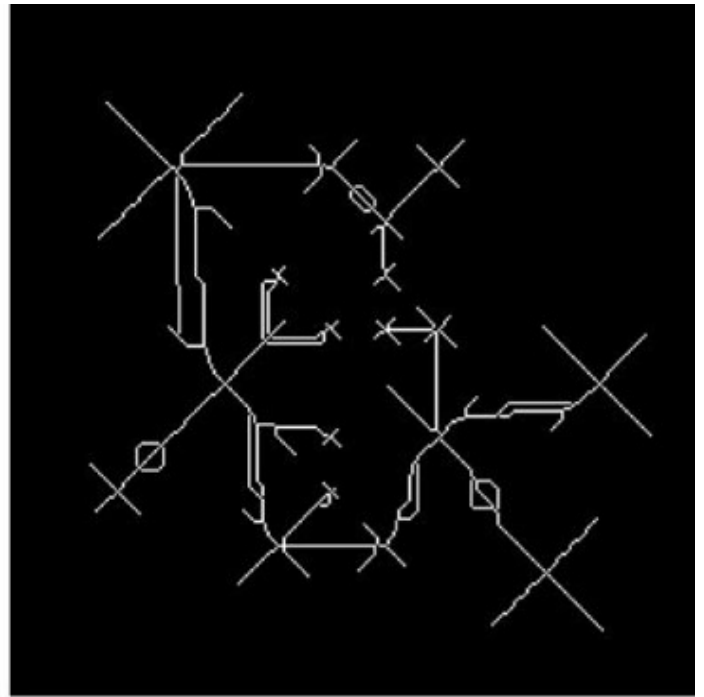
The result is shown in Figure 11.28(a).

Note the differences between these skeletons and those produced by the Zhang-Suen algorithm as shown in Figure 11.25.

Figure 11.28: Examples of Guo-Hall skeletonization



(a)



(b)

Skeletonization Using the Distance Transform

The distance transform can be used to provide the skeleton of a region R . We apply the distance transform, using mask 1, to the image negative, just as we did for the circle previously. Then the skeleton consists of those pixels (i, j) for which

$$d(i, j) \geq \max\{d(i-1, j), d(i+1, j), d(i, j-1), d(i, j+1)\}.$$

For example, suppose we take a small region consisting of a single rectangle, and find the distance transform of its negative:

```
>> c = zeros(7,9); c(2:6,2:8)=1;
>> mask = [Inf 1 Inf;1 0 Inf;Inf Inf Inf];
>> cd=disttrans(1-c,mask1)
```

cd =

0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	0
0	1	2	2	2	2	2	1	0
0	1	2	3	3	3	2	1	0
0	1	2	2	2	2	2	1	0
0	1	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0	0

MATLAB/Octave

The skeleton can be obtained by using MATLAB's `ordfilt2` function, which was introduced in Chapter 5. This can be used to find the largest value in a neighborhood, and the neighborhood can be very precisely defined:

```
>> cd2=ordfilt2(cd,5,[0 1 0;1 1 1;0 1 0])
cd2 =
```

0	1	1	1	1	1	1	1	0
1	1	2	2	2	2	2	1	1
1	2	2	3	3	3	2	2	1
1	2	3	3	3	3	3	2	1
1	2	2	3	3	3	2	2	1
1	1	2	2	2	2	2	1	1
0	1	1	1	1	1	1	1	0

```
>> cd2 <= cd
ans =
```

1	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0
0	0	1	0	0	0	1	0	0
0	0	0	1	1	1	0	0	0
0	0	1	0	0	0	1	0	0
0	1	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	1

MATLAB/Octave

In Python, all of this can be done very similarly. First define the image and compute its distance transform.

```

In : c = np.zeros((7,9)); c[1:6,1:8] = 1
In : inf = float('inf')
In : mask = np.array([[inf,1,inf],[1,0,inf],[inf,inf,inf]])
In : cd = disttrans(1-c,mask)

```

Python

Next the largest elements:

```

In : cd2 = ndi.rank_filter(cd,rank=4,footprint=np.array
    ([[0,1,0],[1,1,1],[0,1,0]]))
In : print (cd2 <= cd)*1

```

Python

We can easily restrict the image so as not to obtain the extra 1's in the corners of the result. Now let's do the same thing with our circles image:

```

>> c = 1-imread('circles2.png');
>> cd = disttrans(c,mask);
>> cd2 = ordfilt2(cd,5,[0 1 0;1 1 1;0 1 0]);
>> imshow((cd2 <= cd) & ~c)

```

MATLAB/Octave

or

```

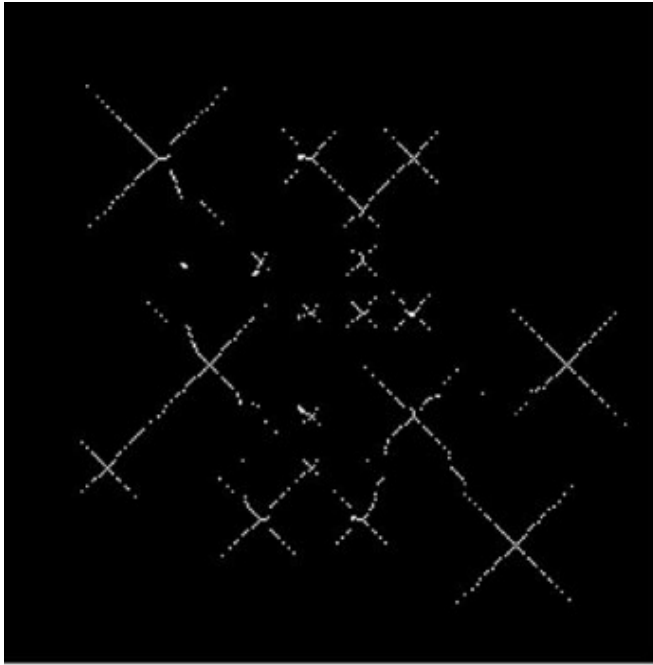
In : c = 1 - io.imread('circles2.png')/255
In : cd = disttrans(c,mask)
In : cd2 = ndi.rank_filter(cd,rank=4,footprint=np.array
    ([[0,1,0],[1,1,1],[0,1,0]]))
In : io.imshow((cd2 <= cd)*(1-c))

```

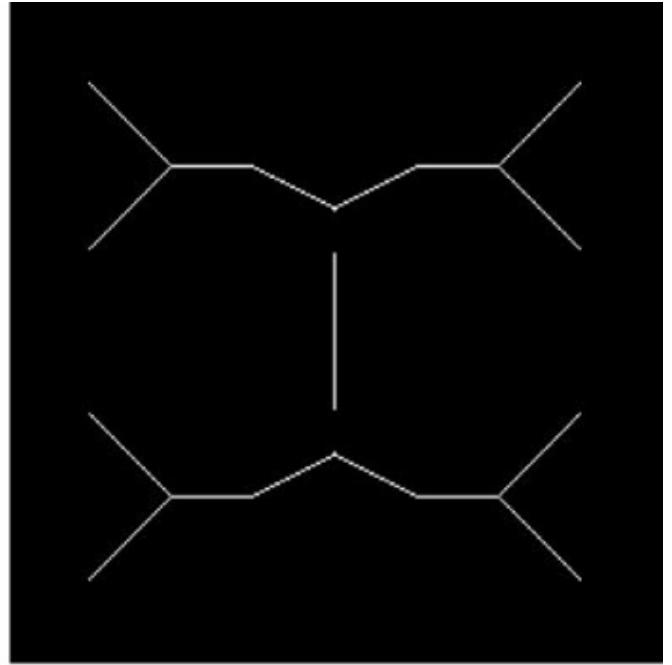
Python

and the result is shown in Figure 11.29(a). Image (b) shows the distance transform applied to the central image in Figure 11.14. The use of the commands `(cd2 <= cd) & ~c` or `(cd2 <= cd)*(1-c)` blocks out the outside of the circles, so that just the skeleton is left. We will see in Chapter 10 how to thicken this skeleton, and also other ways of obtaining the skeleton.

Figure 11.29: Skeletonization using the distance transform



(a) Circles skeleton



(b) Skeleton from Figure 11.14

Details about both the Guo-Hall and Zhang-Suen algorithms, and others, with discussions and examples, can be found in [33].

11.9 Programs

First, programs in MATLAB/Octave for the distance transform:

```
function res=disttrans(image,mask)
%
% This function implements the distance transform by applying MASK to
% IMAGE, using the two step algorithm with "forward" and "backwards" masks
%
%
backmask = rot90(rot90(mask));
[mr,mc] = size(mask);
if (mod(mr,2)==0 || mod(mc,2)==0) then
    error('The_mask_must_have_odd_dimensions.')
end;
[r,c] = size(image);
```

MATLAB/Octave

```

nr = (mr-1)/2;
nc = (mc-1)/2;
image2 = padarray(image,[nr,nc],'replicate');

%
% This is the first step; replacing R values with 0 and other values
% with infinity
%
image2(find(image2==0))=Inf;
image2(find(image2==1))=0;
%
% Forward pass
%
for i=nr+1:nr+r,
    for j=nc+1:nc+c,
        temp = image2(i-nr:i+nr,j-nc:j+nc)+mask;
        image2(i,j) = min(temp(:));
    end;
end;
%
% Backward pass
%
for i=r+nr:-1:nr+1,
    for j=c+nc:-1:nc+1,
        temp = image2(i-nr:i+nr,j-nc:j+nc)+backmask;
        image2(i,j) = min(temp(:));
    end;
end;
%
res=image2(nr+1:r+nr,nc+1:c+nc);
end

```

MATLAB/Octave

The Zhang-Suen skeletonization algorithm:

```

function out = zs(image)
    lut_odd = makelut(@zs_odd,3);
    lut_even = makelut(@zs_even,3);
    previous = image;
    current = image & ~applylut(image,lut_odd);
    while true,
        current = current & ~applylut(current,lut_even);
        if isequal(previous,current),
            break;
        end
        current = current & ~applylut(current,lut_odd);
        if isequal(previous,current),
            break;
        end
    previous = current;
    end
    out = current;
end

```

MATLAB/Octave

```

function out = zs_odd(x)
    t = x;
    t(5) = 0;
    B = sum(t(:));
    X = max(max(bwlabel(t,4)));
    out = (B>=2) & (B<=6) & (X==1);
    out = out & (t(6)==0 | t(8)==0 | (t(2)==0 & t(4)==0));
end

function out = zs_even(x)
    t = x;
    t(5)=0;
    B = sum(t(:));
    X = max(max(bwlabel(t,4)));
    out = (B>=2) & (B<=6) & (X==1);
    out = out & (t(2)==0 | t(4)==0 | (t(6)==0 & t(8)==0));
end

```

MATLAB/Octave

and the Guo-Hall skeletonization algorithm:


```

function out = gh(image)
    gh_lut = makelut(@gh_fun,3);
    [r,c] = size(image);
    image2 = repmat([1 2;2 1],r/2,c/2).*image;
    prev = image2;
    g = applylut(image2>0, gh_lut);
    curr = prev .* ~((prev==1) & (g==1));
    while true,
        g = applylut(curr>0, gh_lut);
        curr = curr .* ~((curr==2) & (g==1));
        g = applylut(curr>0,gh_lut);
        curr = curr .* ~((curr==1) & (g==1));
        if isequal(prev,curr),
            break;
        end
        prev = curr;
    end
    out = curr>0;
end

function out = gh_fun(x)
    n = x;
    n(5) = 0;
    B = sum(n(:));
    X = max(max(bwlabel(n,8)));
    S = n(2)*n(4)*n(6)*n(8);
    out = (B>=2 & X==1 & S==0);
end

```

MATLAB/Octave

In Python, a distance transform program is

```

def disttrans(image,mask):
    backmask = np.rot90(np.rot90(mask))
    mr,mc = mask.shape
    if mr%2 == 0 or mc%2 == 0:
        raise ValueError("Error:_Mask_must_have_odd_numbers_of_rows_and_
            columns")
    r,c = image.shape
    nr = (mr-1)/2
    nc = (mc-1)/2
    image2 = np.float64(np.pad(image,[nr,nc],mode='edge'))
    image2[np.where(image2==0)]=float('inf')
    image2[np.where(image2==1)]=0
    for i in range(r):
        for j in range(c):
            image2[i+nr,j+nc] = (image2[i:i+mr,j:j+mc]+mask).min()
    for i in range(r)[::-1]:
        for j in range(c)[::-1]:
            image2[i+nr,j+nc] = (image2[i:i+mr,j:j+mc]+backmask).min()
    return image2[nr:nr+r,nc:nc+c]

```

Python

A program for Zhang-Suen skeletonization:

```

from numpy import array_equal,copy,reshape
from scipy.ndimage import generic_filter as gfilt
from skimage.measure import label as bwlabel

def zs(im):
    prev = copy(im)
    curr = prev*(1-gfilt(prev,zs_fun,size=(3,3),extra_arguments=(1,)))
    while True:
        curr = curr*(1-gfilt(curr,zs_fun,size=(3,3),extra_arguments=(0,)))
        curr = curr*(1-gfilt(curr,zs_fun,size=(3,3),extra_arguments=(1,)))
        if array_equal(prev,curr):
            break
        else:
            prev = copy(curr)
    return curr

def zs_fun(x,parity):
    n = np.reshape(x,[3,3])
    n[1,1] = 0
    B = n.sum()
    X = bwlabel(n,background=0).max()+1
    out = (B>=2) and (B<=6) and (X==1)
    if parity==0:
        out = out & (x[1]==0 or x[3]==0 or (x[5]==0 and x[7]==0))
    elif parity==1:
        out = out & (x[5]==0 or x[7] ==0 or (x[1]==0 and x[3]==0))
    return out

```

Python

And a program for Guo-Hall skeletonization:

```

from numpy import reshape,zeros,copy,array_equal,array,tile
from scipy.ndimage import generic_filter as gfilt
from skimage.measure import label as bwlablel

def gh(im):
    r,c = im.shape
    im2 = tile(array([[1,2],[2,1]]),[r/2,c/2])*im
    prev = copy(im2)
    curr = prev*(1-gfilt(prev,gh_fun,size=(3,3),extra_arguments=(1,)))
    while True:
        curr = curr*(1-gfilt(curr,gh_fun,size=(3,3),extra_arguments=(2,)))
        curr = curr*(1-gfilt(curr,gh_fun,size=(3,3),extra_arguments=(1,)))
        if array_equal(prev,curr):
            break
        else:
            prev = copy(curr)
    return curr

def gh_fun(x,parity):
    n = (reshape(x,[3,3])>0)*1
    n[1,1] = 0
    B = n.sum()
    X = bwlablel(n,8,background=0).max()+1
    return ((B>=2) and (X==1) and (x[1]*x[3]*x[5]*x[7]==0) and (x[4]==
        parity))

```

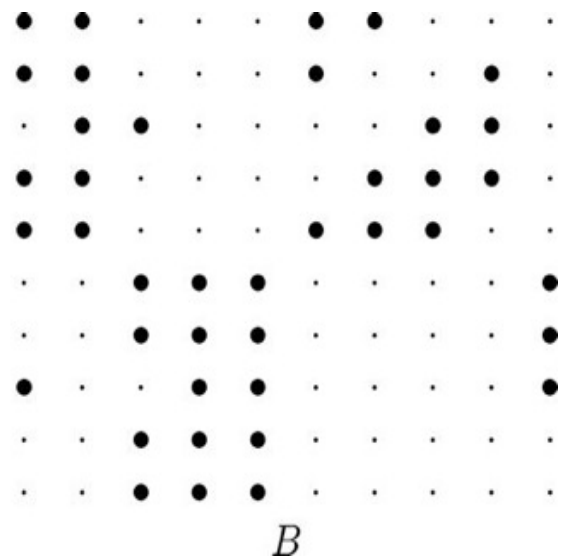
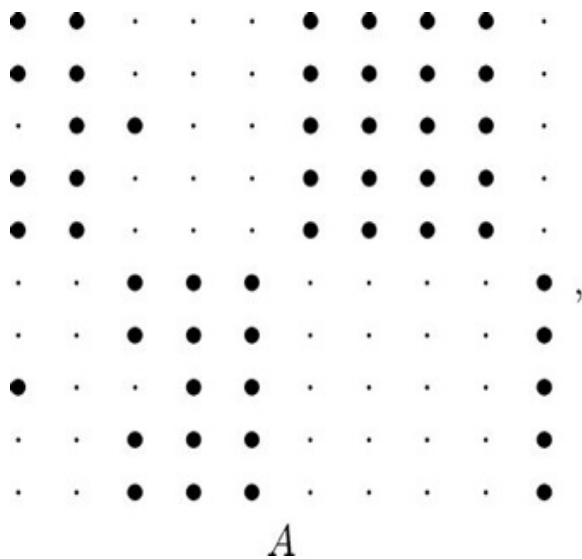
Python

Exercises

1. What are the coordinates of the 4-neighbors of the pixel (i,j) ? What are the coordinates of its 8-neighbors?
2. Find the length of the shortest 4-path from
 - (a) pixel $(1, 1)$ to pixel $(5, 4)$
 - (b) pixel $(3, 1)$ to pixel $(1, 6)$
 - (c) pixel (i, j) to pixel (l, m)

For this question, if a path between pixels p and q consists of $p = p_0, p_1, p_2, \dots, p_n = q$ where all of the pixels in the path are different, then the length of that path is defined to be $n - 1$.

3. Find the shortest 8-paths between each pair of pixels in the preceding question.
4. Consider the two images



Find the 4-components and 8-components of each image.

5. The above matrices were obtained with the MATLAB/Octave commands

```
>> A=magic(10)>50
>> B=magic(10)>60
```

MATLAB/Octave

If you use Python you will have to enter the matrices manually. Check your answers of the previous question with the `bwlabel` function of MATLAB/Octave or `sk.measure.label` in Python.

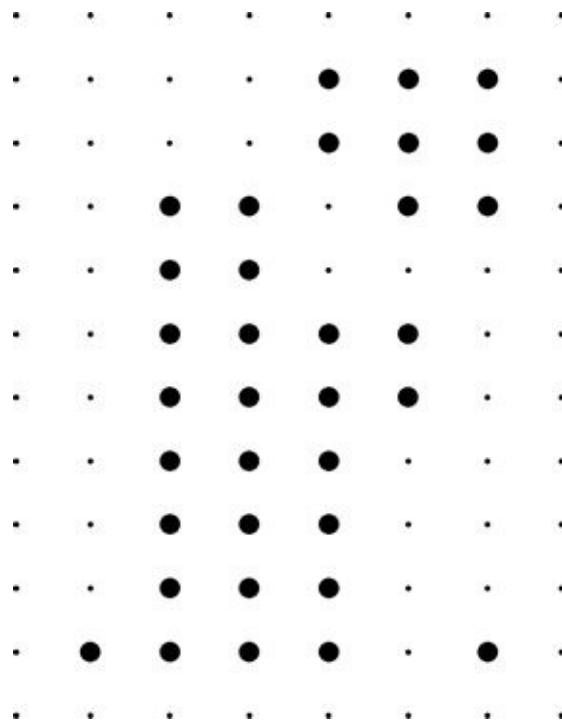
6. We can define the *6-neighbors* of a pixel p with coordinates (x, y) to be the pixels with coordinates $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, $(x, y - 1)$, $(x + 1, y + 1)$, and $(x - 1, y - 1)$. Draw a diagram showing the six 6-neighbors of a pixel.
7. Prove the triangle inequality for the Euclidean distance metric, and for the 4-path and 8-path metrics d_4 and d_8 .
8. Define the relation *6-connectedness* as:
 p is 6-connected to q if there is a path $p = p_1, p_2, \dots, p_n = q$ such that for each $i = 1, 2, \dots, n - 1$, p_i is 6-adjacent to p_{i+1} .
 Show that this is an equivalence relation.
9. Find the lengths of the shortest 6-paths between pixels with the following coordinates:

- (a) $(0, 0)$ and $(2, 2)$,
- (b) $(1, 2)$ and $(5, 4)$,
- (c) $(2, 1)$ and $(6, 8)$,
- (d) $(3, 1)$ and $(7, 4)$

Can you develop an expression for a 6-path metric?

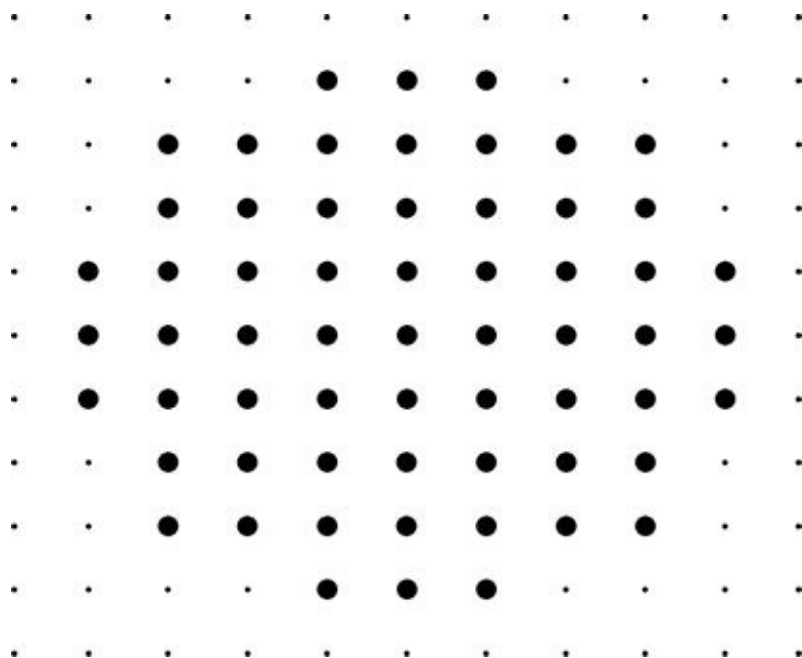
10. Show how to refine the algorithms for component labeling to label the 6-components of a binary image.

- 11. For the following image:



use your algorithm developed in the previous question to label the 6-components.

- 12. Use `bwlabel` or `sk.measure.label` to determine the number of blobs in the example given at the beginning of this chapter. Is there any difference between the results using 4 and 8 adjacency? Can you account for your answer?
- 13. Obtain the 6-components of the images *A* and *B* given in Question 4.
- 14. Let *C* be the image:



Obtain the 4-boundary and 8-boundary of this image.

- 15. The image in the previous question was obtained with the MATLAB commands:

```
>> [x,y] = meshgrid(-5:5, -5:5);
>> C = (x.^2+y.^2)<20
```

MATLAB/Octave

or with the Python commands

```
In : x,y = np.mgrid[-5:6,-5:6]
In : C = ((x**2+y**2)<20)*1
```

Python

Check your answers to Question 14 with the `bwperim` or the `sk.measure.perimeter` function.

- 16. Determine if each of the following foreground pixels is (a) 4-simple, (b) 8-simple.



- 17. Calculate $C_S(p)$ for each configuration in the previous question, and show that the result is equal to $C(p)$ in each case.

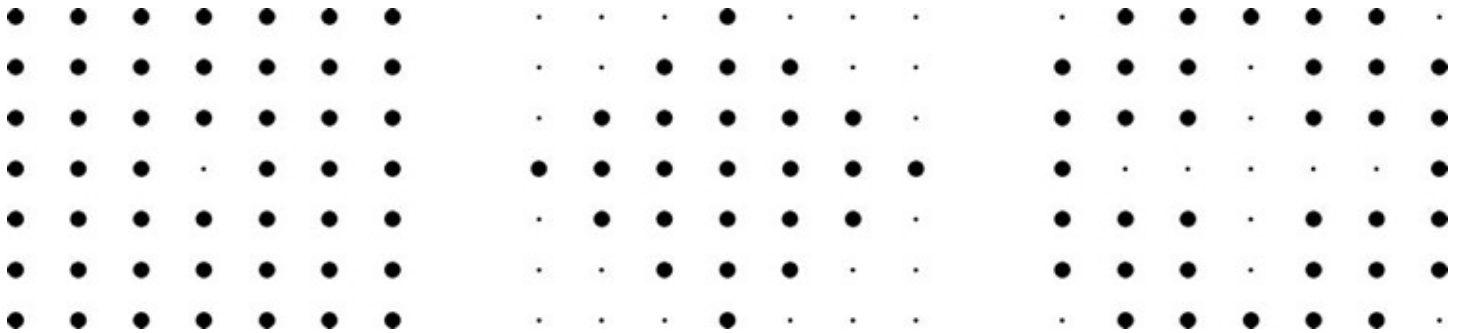
- 18. Find configurations of pixels surrounding p for which

(a) $C(p) = A(p) - 2$

(b) $C(p) = A(p) - 3$

(c) $C(p) = A(p) - 4$

- 19. Show that $C(p) \leq A(p)$ for every configuration of pixels.
- 20. Skeletonize each of the following images using (a) the Zhang-Suen algorithm, and (b) the Guo-Hall algorithm:



- 21. Consider the cross-shaped object formed by starting with an 11×11 square and removing a 3×3 square from each corner. Skeletonize it by each of the algorithms.
- 22. Check your answers to the previous two questions with your computer system.
- 23. Which of the algorithms seems to be the fastest, in requiring the least number of iterations, or removing the greatest number of pixels at each step?
- 24. Sketch the medial axes of: (a) a 2×1 rectangle, and (b) a triangle.
- 25. Apply both algorithms to the image `circles2.png`.

(a) Which one works faster?

(b) Which gives the best looking results?

- 26. Repeat the previous question on some other binary images.
- 27. For each of the following images:

0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0
0	1	1	1	1	0	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	1	1
0	1	1	1	1	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	1	0	0
0	1	1	1	1	1	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0

apply the distance transform to approximate distances from the region containing 1's to all other pixels in the image, using the masks:

- (i)

		1		
1	0	1		
	1			
1	1	1		
	1	0	1	
- (ii)

1	1	1
4	3	4
3	0	3
- (iii)

4	3	4		
	11		11	
11	7	5	7	11
	5	0	5	
11	7	5	7	11
- (iv)

		11		
	11		11	

and applying any necessary scaling at the end.

- 28. Apply the distance transform and use it to find the skeleton in the images `circles2.png` and `nicework.png`.
- 29. Compare the result of the previous question with the results given by the Zhang-Suen and Guo-Hall methods. Which method produces the most visually appealing results? Which method seems fastest?

12 Shapes and Boundaries

12.1 Introduction

In this chapter we shall investigate tools for examining image shapes. Some tools have already been discussed in Chapter 10; now we look at specific methods for examining shapes of objects. Questions we might ask about shapes include:

- How do we tell if two objects have the same shape?
- How can we classify shape?
- How can we describe the shape of an object?

Formal means of describing shapes are called *shape descriptors*. Shape descriptors may include size, symmetry, and length of perimeter. A precise definition of the exact shape in some efficient manner is a *shape representation*.